

时序逻辑电路

引言

本章将在组合逻辑电路和锁存器、触发器的基础上讨论时序电路。在第4章所讨论的组合逻辑电路中,任一时刻的输出信号仅仅由该时刻的输入信号所决定,而时序电路在任一时刻的输出信号不仅与当时的输入信号有关,而且与电路原来的状态有关。也就是说,时序电路中除具有逻辑运算功能的组合电路外,还必须具有能够记忆电路状态的存储单元或延迟单元,这些存储或延迟逻辑单元主要由锁存器或触发器组成。

本章首先介绍时序电路的基本概念,然后讨论其分析与设计方法。其中,重点侧重于同步时序电路的分析与设计。在明确基本概念和基本方法之后,介绍逻辑设计中常用的典型时序电路模块,以及简单的时序可编程逻辑器件 GAL。最后通过简单实例介绍用 Verilog 描述时序电路的方法。

6.1 时序逻辑电路的基本概念

日常生活中,时序逻辑的实例并不鲜见,例如楼房电梯的控制便是一个典型的时序逻辑问题。电梯的控制电路需要根据电梯内和各楼层入口处的按键信号,以及电梯当前的状态来决定电梯的升降,同时将电梯当前所处楼层信号输出到电梯内外。这里,按键信号和所到达的楼层信号是时序逻辑的“输入信号”,升降信号和到达楼层的显示则是其“输出信号”。显然,控制电路中必须具有存储单元,以记忆当前电梯所在的楼层。可以定义电梯目前所处楼层为现在的状态,简称“现态”,将要到达的楼层为下一个状态,简称“次态”,楼层的变换即为“状态转换”。电梯的升降不仅取决于当前各按键的输入信号,而且取决于电梯运转的历史状态。例如,电梯正在从低楼层向高楼层上升,若这时电梯内已有人按下更高楼层的按键,或更高的楼层有人召唤,电梯则应向高一层转换其状态而暂时忽略电梯内要求下车的按键输入或低楼层的召唤信号。这种确定电梯状态如何转换的信号称为“激励信号”。

诸如上例中的输入信号、输出信号、激励信号以及现态、次态及其状态转换的关系是时序电

路研究的主要内容。

6.1.1 时序逻辑电路的基本结构与分类

1. 时序电路的基本结构

时序电路的基本结构如图 6.1.1 所示,它由完成逻辑运算的组合电路和起记忆作用的存储电路两部分构成,其中,存储电路由触发器或锁存器组成。为了方便,图中各组逻辑变量均以向量形式表示,其中, $I=(I_1, I_2, \dots, I_l)$ 为输入信号, $O=(O_1, O_2, \dots, O_j)$ 为输出信号, $E=(E_1, E_2, \dots, E_k)$ 为驱动存储电路转换为下一状态的激励信号,而 $S=(S_1, S_2, \dots, S_m)$ 为存储电路状态,称为状态信号,亦称为状态变量,它表示时序电路当前的状态,简称现态。状态变量 S 被反馈到组合电路的输入端,与输入信号 I 一起决定时序电路的输出信号 O ,并产生对存储电路的激励信号 E ,从而确定电路的下一状态,即次态。于是,上述 4 组变量间的逻辑关系可用下列三个向量函数形式的方程来表达:

$$E=f(I, S) \quad (6.1.1)$$

$$S^{n+1}=g(E, S^n) \quad (6.1.2)$$

$$O=h(I, S) \quad (6.1.3)$$

式(6.1.1)表达了激励信号与输入信号、状态变量的关系,称为时序电路的激励方程。式(6.1.2)表达了存储电路从现态到次态的转换,故称为状态转换方程,简称转换方程。其中, S^n 和 S^{n+1} 分别表示存储电路的现态和次态。而式(6.1.3)表达了时序电路的输出信号与输入信号、状态变量的关系,称为输出方程。上述三个向量函数形式的方程分别对应于表达时序电路的三个基本方程组:激励方程组、转换方程组和输出方程组。

如上所述,时序电路是状态依赖的,故又称为状态机(State Machine, SM)。本章将只限于讨论有限数量的存储单元构成的状态机,因而其状态数也是有限的,称为有限状态机(Finite State Machine, FSM)。

由上述基本结构和方程可见,时序电路具有以下主要特征:

- ① 时序电路由组合电路和存储电路组成。
- ② 时序电路的状态与时间因素相关,即时序电路在任一时刻的状态变量不仅是输入信号的函数,而且还是电路以前状态的函数,并由当前输入变量和状态决定电路的下一状态。
- ③ 时序电路的输出信号由输入信号和电路状态共同决定。

2. 异步与同步时序电路

时序电路可分为异步时序电路和同步时序电路两大类。

若电路中触发器的时钟输入端没有连接在统一的时钟脉冲上,或电路中没有时钟脉冲(如 SR 锁存器构成的时序电路),从而,电路中各存储单元的状态更新不是同时发生的,则这种电路称为异步时序电路。根据电路是对脉冲边沿敏感还是对电平敏感,异步时序电路又分为脉冲异

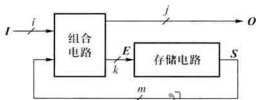


图 6.1.1 时序电路的基本结构

步时序电路(由触发器构成)和**电平异步时序电路**(由锁存器构成)两种。异步时序电路的状态转换取决于以任意时间间隔变化的输入信号序列,若有多个信号输出,其存储电路的状态转换因存在时间差异而造成短时间输出状态的不稳定,而且这种不稳定的状态有时是不容易判定的,常常给电路设计和调试带来困难。

与异步时序电路不同,**同步时序电路**中存储电路状态的转换是在同一时钟脉冲源的同一边沿作用下同步动作的,它也称作**时钟同步状态机**(Clocked Synchronous SM),其结构如图 6.1.2 所示。同步时序电路的存储电路一般用触发器实现,所有触发器的时钟输入端都应接在同一个时钟脉冲源上,而且它们的时钟脉冲触发沿也都应一致。因此,所有触发器的状态更新是在同一时刻,其输出状态变换的时间差异很小。在时钟脉冲两次作用的间隔期间,从触发器输入到状态输出的通路被切断,即使此时输入信号发生变化,也不会改变各触发器的输出状态,所以较少发生因状态转换不同步而引起的输出状态不稳定的现象。更重要的是,其电路的状态很容易用固定周期的时钟脉冲边沿清楚地分离为序列步进。其中,每一个步进都可以通过输入信号和所有触发器的现态单独进行分析,从而有一套较系统、易掌握的分析和设计方法,电路行为也很容易用 HDL 来描述。所以,目前结构较复杂、运行速度较高的时序电路广泛采用同步方式来实现。很多大规模可编程逻辑器件的应用电路和专用集成电路的设计,也都采用同步时序电路的结构。

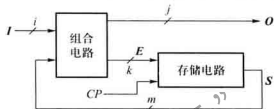


图 6.1.2 同步时序电路的结构

3. 米利型和穆尔型时序电路

米利 (Mealy) 型时序电路的结构如图 6.1.3 所示,事实上是将图 6.1.2 中的组合电路拆分成激励组合电路和输出组合电路两部分。米利型时序电路的输出信号 O 是状态变量 S 和输入信号 I 二者的函数,即 $O=h(I,S)$ 。这种时序电路在时钟脉冲的两个触发沿之间,输出信号随时可能受到非时钟同步的输入信号作用而发生变化,从而影响电路输出的同步性。

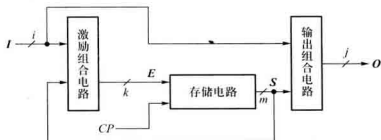


图 6.1.3 米利型时序电路的结构

穆尔 (Moor) 型时序电路是米利型时序电路的一种特例,它的输出信号 O 仅仅是状态变量 S 的函数,即 $O=h(S)$,其结构如图 6.1.4 所示。穆尔型时序电路的输出信号只取决于与时钟同步

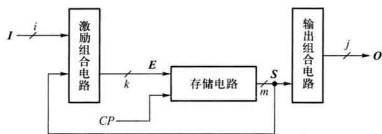


图 6.1.4 穆尔型时序电路的结构

的各触发器的状态,在时钟脉冲触发沿的间隔期间,不受非同步的输入信号影响。实际上,米利型时序电路中有时也有一个或多个输出可能是穆尔型的,即它们的输出只取决于触发器的状态。

在现代高速时序电路设计中,一般尽量采用穆尔型时序电路结构,以利于后续高速电路的同步。在米利型时序电路的输出端增加一级存储电路,构成“流水线输出”形式,是将其转化为穆尔型电路的最简单的方法,电路结构如图 6.1.5 所示。需要注意的是,流水线存储电路将把输出信号延迟一个时钟周期。虽然流水线输出电路增加了一些逻辑元件,但它的输出信号同步特性更好,具有更好的稳定性和抗干扰性能。在大规模集成电路技术成熟的今天,节省逻辑元件的数量经常不再是逻辑电路设计者追求的唯一目标,电路工作的稳定性和可靠性,以及工作速度的提高才是更重要的要求。

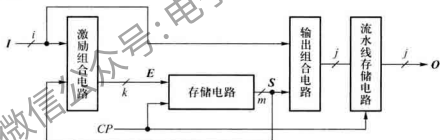


图 6.1.5 流水线输出的米利型时序电路的结构

6.1.2 时序逻辑电路功能的表达

时序电路的功能可用逻辑方程组、转换表、状态表、状态图和时序图等形式来表达,也可以用 HDL 语言描述。从理论上讲,有了激励方程组、转换方程组和输出方程组,时序电路的功能就被唯一地确定了。但是,对于许多时序电路而言,仅从这三组方程还不易判断其逻辑功能,在设计时序电路时,也往往很难根据给出的逻辑需求直接写出这三组方程。因此,还需要用能够直观反映电路状态变化序列全过程的转换表、状态表和状态图来帮助。三组方程、转换表、状态表和状态图之间可以实现相互转换,根据其中任意一种表达方式,都可以画出时序图。下面通过实例介绍时序电路逻辑功能的前五种表达方法,使用 Verilog HDL 描述时序电路的方法将在 6.7 节讨论。

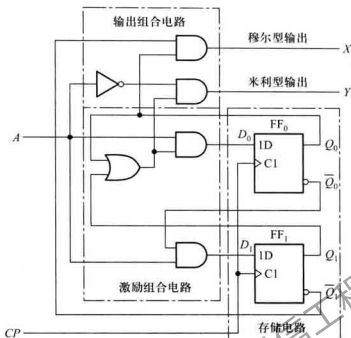


图 6.1.6 同步时序电路一例

考虑图 6.1.6 所示电路,它由组合电路与存储电路两大部分组成。其中,存储电路由两个 D 触发器 FF_1 、 FF_0 构成,二者共用一个时钟信号 CP ,从而构成一个同步时序电路。组合电路又分为激励电路和输出电路两部分。电路的输入信号为 A ,输出信号为 X 、 Y 。对触发器的激励信号分别为 D_1 和 D_0 , Q_1 、 Q_0 为电路的状态变量。从图中可以看出,输出信号 Y 是状态变量 Q_1 、 Q_0 和输入信号 A 的函数,所以从总体上看,这是一个米利型时序电路,但是,输出信号 X 纯粹由状态变量 Q_1 、 Q_0 决定,电路中又存在一个穆尔型输出 X 。

1. 逻辑方程组

(1) 激励方程组

根据图 6.1.6 中的组合电路,可写出对两个 D 触发器的激励方程组

$$D_0 = (Q_1 + Q_0)A \quad (6.1.4)$$

$$D_1 = \overline{Q_0}A \quad (6.1.5)$$

(2) 转换方程组

将式(6.1.4)和式(6.1.5)分别代入 D 触发器的特性方程 $Q^{n+1} = D$,于是,得到转换方程组

$$Q_0^{n+1} = (Q_1^n + Q_0^n)A \quad (6.1.6)$$

$$Q_1^{n+1} = \overline{Q_0^n}A \quad (6.1.7)$$

以上两式表明,触发器的次态 Q_0^{n+1} 和 Q_1^{n+1} 是输入变量 A 和触发器现态 Q_0^n 、 Q_1^n 的函数。

(3) 输出方程组

图 6.1.6 的逻辑图中有两个输出变量 X 、 Y ，可根据输出组合电路得到输出方程组

$$X = \overline{Q_1} Q_0 \quad (6.1.8)$$

$$Y = (Q_1 + Q_0) \overline{A} \quad (6.1.9)$$

显然， X 是穆尔型输出， Y 是米利型输出。

上述三组方程中，激励方程组和输出方程组表达了时序电路中全部组合电路的特性，而转换方程组则表达了存储电路从现态到次态的状态转换特性。转换方程两边的状态变量，分别以上标 n 表示现态，以上标 $n+1$ 表示次态，以区别这两种不同的状态。

2. 转换表

与组合电路分析方法类似，根据逻辑表达式 (6.1.6)、(6.1.7) 和式 (6.1.8)、(6.1.9) 可以列出真值表如表 6.1.1 所示。真值表的输入变量为 Q_1^n 、 Q_0^n 和 A ，输出变量为 Q_1^{n+1} 、 Q_0^{n+1} 和 X 、 Y 。由于该真值表反映了触发器从现态到次态的转换，故称为**状态转换真值表**。一般来说，有 m 位状态变量和 i 位输入信号，就存在 2^{m+i} 种状态-输入组合，真值表就应有 2^{m+i} 行。

表 6.1.1 图 6.1.6 电路的状态转换真值表

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	X	Y
0	0	0	0	0	0	0
0	0	1	1	0	0	0
0	1	0	0	0	1	1
0	1	1	0	1	1	0
1	0	0	0	0	0	1
1	0	1	1	1	0	0
1	1	0	0	0	0	1
1	1	1	0	1	0	0

在分析和设计时序电路时，更常用的是**转换表**，如表 6.1.2 所示。它与表 6.1.1 完全等效，但形式更紧凑明了。表 6.1.2 用矩阵形式表达出在不同现态和输入条件下，电路的状态转换和输出逻辑值。表中，输出信号 X 是穆尔型输出，故将其与现态 $Q_1^n Q_0^n$ 对应的逻辑值单列一栏；输出信号 Y 的逻辑值不仅取决于 Q_1^n 和 Q_0^n ，而且会跟随 A 变化，所以表示在 $A=0$ 和 $A=1$ 两栏的斜线后面。需要注意的是，虽然它与斜线前的次态 $Q_1^{n+1} Q_0^{n+1}$ 列在一起，但仍是现态 $Q_1^n Q_0^n$ 和输入 A 的函数。

表 6.1.2 图 6.1.6 电路的转换表

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$		X
	$A=0$	$A=1$	
00	00/0	10/0	0
01	00/1	01/0	1
10	00/1	11/0	0
11	00/1	01/0	0

3. 状态表

转换表以各触发器逻辑值的编码表示时序电路的状态。在分析一个电路时,给每个编码状态分别赋予一个具体名称有时更便于与实际问题结合进行分析和记忆;时序电路设计过程中,在尚未进行状态分配之前,也必须首先给各个状态命名,以表达状态之间的转换关系。在本例中,可以简单地命名4个状态为 $00=a$, $01=b$, $10=c$, $11=d$ 。当然,也可以用意义更明显的中、英文单词或数字来对状态命名。将表 6.1.2 中 $Q_1 Q_0$ 的状态用其状态名代替,则得到表 6.1.3 所示的状态表。表中, S^n 表示现态, S^{n+1} 表示次态,斜线后的逻辑值为输出变量 Y 的值。

和转换表相比,使用状态表更容易理解时序电路的行为和结果。如果状态命名合理,即使较复杂的时序电路,直接通过状态名就可以得到各状态的实际意义和转换关系。但是,状态表中的状态变量没有标明编码,与时序电路的3个逻辑方程组及实际电路图难以联系,这方面的信息少于转换表。转换表和状态表虽然形式和内容相似,然而在应用上是有差别的。

表 6.1.3 图 6.1.6 电路的状态表

S^n	S^{n+1} / Y		X
	$A=0$	$A=1$	
a	$a/0$	$c/0$	0
b	$a/1$	$b/0$	1
c	$a/1$	$d/0$	0
d	$a/1$	$b/0$	0

4. 状态图

将表 6.1.3 的状态表转换为如图 6.1.7(a)所示的状态图,可以更直观形象地表示出时序电路运行中的全部状态、各状态间相互转换的关系以及转换的条件和结果。图中,每一个圆圈都对应着一个状态,圆圈中标出状态名;每一个带箭头的方向线都表示一个转换,箭头指示出状态转换的方向。当方向线的起点和终点都在同一个圆圈上时,则表示状态不变。引起该状态转换的输入变量逻辑值标在方向线旁斜线左侧,如图中 A 值。米利型输出变量的逻辑值标在方向线旁

斜线右侧,是本次状态转换前的逻辑值,如图中 Y 值。它由方向线起点的状态和斜线前的输入变量共同决定。而穆尔型输出变量的逻辑值则标在圆圈内的状态名后,因为状态一旦确定,其输出值随之确定,如图中 X 值。当设计时序电路时,首先需要画出这种形式的状态图,以明确状态的数目、状态转换的方向以及状态转换的条件和相应的输出信号。

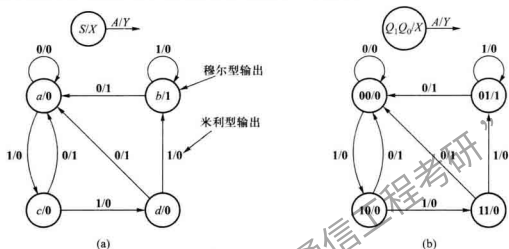


图 6.1.7 图 6.1.6 所示同步时序电路的状态图
(a) 与状态表对应的状态图 (b) 与转换表对应的状态图

状态图也可以与转换表相对应,如图 6.1.7(b)所示,圆圈中以二进制编码表示状态,在分析时序电路时可先得到这种形式的状态图,然后再进一步研究其功能。

需要强调指出,图 6.1.7(a)或(b)中状态转换的方向,取决于电路中下一个时钟脉冲触发沿到来前瞬间的输入信号。如果在此之前输入信号发生改变,则状态转换的方向也会立即改变。例如图 6.1.7(a)中,当处于状态 c 时,如果输入 A 保持为 1 ,则输出 Y 为 0 ,下一状态将转换为 d ;若在下一个时钟脉冲触发沿到来之前, A 由 1 变化为 0 ,则 Y 立即变化为 1 ,下一状态将转换为 a 。

绘制状态图时需要注意不能漏画方向线。原则上,以某一状态为起点的方向线数量应为 2^i 根, i 为电路输入变量的数目,即输入变量的每一种组合应当对应 1 根方向线。例如,在图 6.1.7的状态图中只有一个输入变量 A ,共有 2^1 种组合: $A=0$ 和 $A=1$,所以从每个状态都引出两根方向线。

当出现输入变量不同而方向线的起点和终点一致时,可以合并方向线。例如 5.5.2 小节中讨论的 JK 触发器状态图,它有两个输入变量 J 、 K ,以每个状态为起点的方向线原应当有 $2^2=4$ 根,经合并后仅余两根,如图 5.5.3 所示,从而使状态图得到简化。以图中触发器状态 $Q=0$ 为例,转换到状态 $Q=1$ 的方向线原有两根,输入变量的组合分别为 $JK=10$ 和 $JK=11$,即同一方向上 K 是无关变量,于是可将两根方向线合并为一根,输入变量表示为 $JK=1\times$ 。其余方向线的合并类推,最后得到简化的状态图。

因为人类对图形的理解能力远高于相应的表格、文字和公式,所以状态图在日益复杂的时序

电路设计中愈显重要。从状态图出发,可以直接列出状态表,进而列出转换表。EDA 软件一般都可以以某种形式接受转换表的输入,有些软件甚至能以某种形式接受状态表或状态图的输入。这样,在 6.3 节所讨论的同步时序电路的设计,从状态图以后的很多工作都可以由软件自动完成,状态图的描绘才是一项真正具有挑战性的工作。

5. 时序图

与组合电路一样,波形图能直观地表达时序电路中各信号在时间上的对应关系,通常把时序电路的状态、输出对输入信号(包括时钟信号)响应的波形图称为**时序图**。它不仅便于电路调试时检查逻辑功能、排查故障或差错,而且在运用 HDL 设计电路时可用于电路的仿真。从逻辑方程组、转换表或状态图都可以导出时序图。图 6.1.6 所示时序电路的时序图如图 6.1.8 所示。

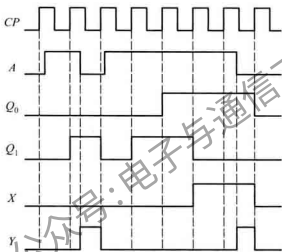


图 6.1.8 图 6.1.6 电路的时序图

使用时序图时要注意,有时它并不完全表达出电路状态转换的全部过程,而是根据需要仅画出部分典型的波形图,如图 6.1.8 中就没有表达出当状态 Q_1Q_0 为 11 而输入 A 为 0 时的状态转换波形和输出波形。

复习思考题

- 6.1.1 时序电路由哪几部分组成? 它和组合电路在逻辑功能和结构上有什么区别?
- 6.1.2 异步时序电路与同步时序电路有哪些不同的特性?
- 6.1.3 表达时序电路的逻辑功能有哪几种方法? 哪种方法可直接从逻辑图得出? 哪几种方法可直接显示出时序电路状态转换的全部过程? 哪种方法可观察到电路的输入信号、触发器状态和输出信号的波形并在实验和 HDL 仿真时应用?
- 6.1.4 试从表 6.1.1 的状态转换真值表推导出该例的激励方程组、转换方程组和输出方程组。
- 6.1.5 试从图 6.1.7(a) 和 (b) 所示状态图分别推导其状态表和转换表。

6.1.6 米利型和穆尔型时序电路的输出特性有何不同? 米利型输出和穆尔型输出在转换表、状态表和状态图中的表达有何不同?

6.2 同步时序逻辑电路的分析

同步时序电路的分析实际上是一个读图、识图的过程:按照给定的时序电路,通过分析其状态和输出信号在输入变量和时钟作用下的转换规律,理解其逻辑功能和工作特性。下面首先介绍分析同步时序电路的一般步骤,然后通过例题加深对分析方法的理解。

6.2.1 分析同步时序逻辑电路的一般步骤

(1) 根据给定的同步时序电路导出下列逻辑方程组:

① 对每个触发器导出激励方程,组成激励方程组;
② 将各触发器的激励方程代入相应触发器的特性方程,得到各触发器的转换方程,组成转换方程组;

③ 对应每个输出变量导出输出方程,组成输出方程组。

上述①和③推导出同步时序电路中全部组合电路的特性,而②则推导出电路的状态转换特性。

(2) 根据转换方程组和输出方程组,列出电路的转换表或状态表,画出状态图和时序图。

(3) 确定电路的逻辑功能,必要的话,可用文字详细描述。

上述步骤是分析同步时序电路的一般化过程,实际分析中可根据具体情况增、减执行,最终达到明确电路的功能即可。

6.2.2 同步时序逻辑电路分析举例

例 6.2.1 分析图 6.2.1 所示同步时序电路的逻辑功能。

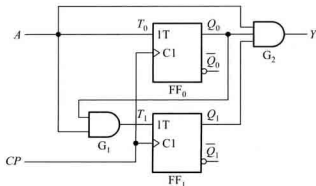


图 6.2.1 例 6.2.1 的逻辑电路图

解：这是一个由两个 T 触发器和两个与门组成的米利型同步时序电路，分析如下。

(1) 根据电路列出三个方程组

① 激励方程组

$$\begin{aligned}T_0 &= A \\T_1 &= A Q_0\end{aligned}$$

② 转换方程组

T 触发器的特性方程为 $Q^{n+1} = T \oplus Q^n = \overline{T} Q^n + T \overline{Q}^n$ ，将两个激励方程分别代入触发器的特性方程，即得转换方程组

$$\begin{aligned}Q_0^{n+1} &= A \oplus Q_0^n \\Q_1^{n+1} &= (A Q_0^n) \oplus Q_1^n\end{aligned}$$

③ 输出方程组

$$Y = A Q_1 Q_0$$

(2) 列出转换表

首先将电路可能出现的现态和输入变量在转换表中列出，本例中需要将 **00, 01, 10, 11** 四个可能的现态列在“ $Q_1^n Q_0^n$ ”栏目中，并把输入 $A=0$ 和 $A=1$ 列在“ $Q_1^{n+1} Q_0^{n+1} / Y$ ”栏目下；然后将现态和输入逻辑值一一代入上述转换方程组和输出方程组，分别求出次态和输出逻辑值。例如，将 $Q_1^n = Q_0^n = A = 0$ 分别代入两个转换方程，得到 $Q_1^{n+1} = 0$ 和 $Q_0^{n+1} = 0$ ；将 $Q_1 = Q_0 = A = 0$ 代入输出方程，得到 $Y = A Q_1 Q_0 = 0$ 。于是可在转换表“ $Q_1^{n+1} Q_0^{n+1} / Y$ ”栏目下， $A=0$ 这一列的第一行填入 **00/0**。其余以此类推，最后列出的转换表如表 6.2.1 所示。

表 6.2.1 例 6.2.1 的转换表

$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$	
	$A=0$	$A=1$
0 0	0 0/0	0 1/0
0 1	0 1/0	1 0/0
1 0	1 0/0	1 1/0
1 1	1 1/0	0 0/1

(3) 画出状态图

由转换表即可画出状态图，如图 6.2.2 所示。

(4) 画出时序图

设电路的初始状态为 $Q_1 Q_0 = 00$ ，根据转换表和状态图，可画出在一系列 CP 脉冲作用下电路的时序图，如图 6.2.3 所示。

(5) 逻辑功能分析

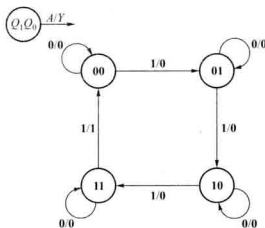


图 6.2.2 例 6.2.1 的状态图

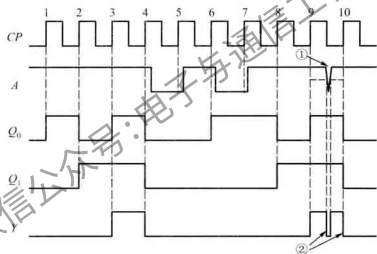


图 6.2.3 例 6.2.1 的时序图

观察状态图和时序图可知,图 6.2.1 的电路是一个由信号 A 控制的可控二进制计数器, CP 为计数脉冲。当 $A=0$ 时停止计数,电路状态保持不变;当 $A=1$ 时,在 CP 上升沿到来后电路状态值加 1,一旦计数到 11 状态, Y 输出 1,且电路状态将在下一个 CP 上升沿回到 00。输出信号 Y 的下降沿可用于触发进位操作。

该电路亦可作为序列信号检测器,用来检测同步脉冲信号序列 A 中 1 的个数,一旦检测到四个 1 状态(这四个 1 状态可以不连续),电路输出 Y 则出现一次从 1 到 0 的跳变。

观察图 6.2.3 的时序图,在第 9 个和第 10 个 CP 脉冲之间,输入信号 A 出现短时间的 0 电平“毛刺”,如图中箭头①所示,结果引起输出 Y 也相应变化。倘若信号 A 的这个“毛刺”是外界干扰造成的(输入信号的引线有时可能较长,易捡拾干扰信号),计数器将输出两次

进位触发脉冲沿,如图 6.2.3 中箭头②所示。如果删除图 6.2.1 中 A 和与门 G_2 输入之间的连线,将电路转化为穆尔型,则能使输出信号 Y 仅取决于电路的状态,其变化始终与时钟同步,而输入信号 A 影响电路状态的时间仅限于 CP 脉冲上升沿前后的瞬间,发生错误的概率将降低到较低的程度,从而提高了电路的抗干扰性能。当然,修改后的电路逻辑功能也将稍有变化。

例 6.2.2 分析图 6.2.4 所示同步时序电路。

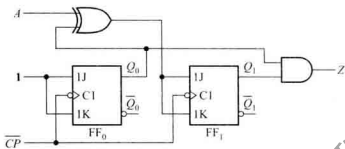


图 6.2.4 例 6.2.2 的逻辑电路图

解: 这是一个由两个下降沿触发的 JK 触发器、一个异或门及一个与门组成的穆尔型同步时序电路。

(1) 根据电路列出三个方程组

① 激励方程组

$$J_0 = K_0 = 1$$

$$J_1 = K_1 = A \oplus Q_0$$

② 转换方程组

将两个激励方程分别代入 JK 触发器的特性方程,得到两个触发器的转换方程

$$Q_0^{n+1} = J_0 \bar{Q}_0^n + \bar{K}_0 Q_0^n = \bar{Q}_0^n$$

$$Q_1^{n+1} = J_1 \bar{Q}_1^n + \bar{K}_1 Q_1^n$$

$$= (A \oplus Q_0^n) \bar{Q}_1^n + A \oplus Q_0^n Q_1^n$$

$$= A \oplus Q_0^n \oplus Q_1^n$$

③ 输出方程组

$$Z = Q_1 Q_0$$

(2) 列出转换表

根据转换方程组和输出方程组可以列出转换表如表 6.2.2 所示。

表 6.2.2 例 6.2.2 的转换表

$Q_1^* Q_0^*$	$Q_1^{*+1} Q_0^{*+1}$		Z
	$A=0$	$A=1$	
00	01	11	0
01	10	00	0
10	11	01	0
11	00	10	1

(3) 画出状态图

根据转换表可以画出状态图如图 6.2.5 所示。

(4) 画出时序图

设电路的初始状态为 $Q_1 Q_0 = 00$, 根据转换表和状态图, 可画出在一系列 $\overline{CP}$ 脉冲作用下的时序图如图 6.2.6 所示。

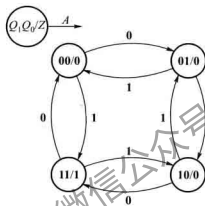


图 6.2.5 例 6.2.2 的状态图

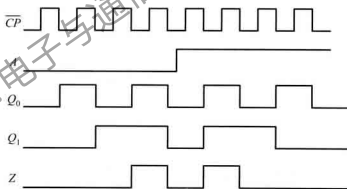


图 6.2.6 例 6.2.2 电路的时序图

(5) 逻辑功能分析

由状态图可以看出, 图 6.2.4 所示电路是一个可逆二进制计数器。当 $A=0$ 时, 进行递增计数, 每来一个时钟脉冲, 计数器值 $Q_1 Q_0$ 加 1, 依次为 $00 \rightarrow 01 \rightarrow 10 \rightarrow 11$ 。每经过 4 个时钟脉冲作用后, 电路的状态循环一次。当 $A=1$ 时, 进行递减计数, 依次为 $11 \rightarrow 10 \rightarrow 01 \rightarrow 00$ 。Z 端在 $Q_1 Q_0$ 为 11 时输出 1。在进行递增计数时, 可以利用 Z 信号的下降沿触发进位操作; 在递减计数时则可用 Z 信号的上升沿触发借位操作。有关计数器的详细内容将在 6.5.2 节讨论。

例 6.2.3 分析图 6.2.7 所示的同步时序电路。

解: 由图 6.2.7 可见, 该同步时序电路没有输入信号, 输出为三个触发器的状态, 是穆尔型时序电路。

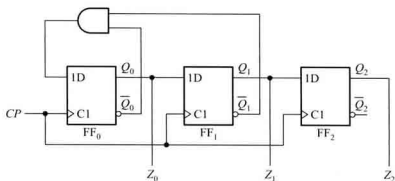


图 6.2.7 例 6.2.3 的逻辑电路图

(1) 根据电路列出逻辑方程组

由于使用 D 触发器, 其特性方程为 $Q^{n+1} = D$, 因此, 可以根据逻辑电路直接列出转换方程组。

① 转换方程组

$$Q_0^{n+1} = D_0 = \overline{Q_1}^n \overline{Q_0}^n$$

$$Q_1^{n+1} = D_1 = Q_0^n$$

$$Q_2^{n+1} = D_2 = Q_1^n$$

② 输出方程组

$$Z_0 = Q_0$$

$$Z_1 = Q_1$$

$$Z_2 = Q_2$$

(2) 列出转换表

由于该电路的输出 Z_1 、 Z_2 、 Z_0 就是各触发器的状态, 所以转换表中可不再单列输出栏。并且电路中没有输入信号, 其转换表可简化为表 6.2.3 所示形式。

表 6.2.3 例 6.2.3 的转换表

$Q_2^n Q_1^n Q_0^n$	$Q_2^{n+1} Q_1^{n+1} Q_0^{n+1}$
0 0 0	0 0 1
0 0 1	0 1 0
0 1 0	1 0 0
0 1 1	1 1 0
1 0 0	0 0 1
1 0 1	0 1 0
1 1 0	1 0 0
1 1 1	1 1 0

(3) 画出状态图

根据转换表可画出电路的状态图如图 6.2.8 所示。由图可见, **001**、**010**、**100** 三个状态形成闭合回路, 电路正常工作时, 其状态总是按照回路中的箭头方向循环变化。这三个状态构成了有效序列, 称它们为**有效状态**, 其余的五个状态则称为**无效状态**。从状态图还可以看出, 无论电路的初始状态如何, 经过若干 CP 脉冲之后, 总能进入有效状态。若电路能从无效状态经一定过程自动进入有效状态, 则称其为具有**自校正能力**。因此, 该电路是具有自校正能力的同步时序电路。

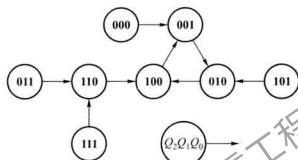


图 6.2.8 例 6.2.3 的状态图

(4) 画出时序图

设电路的初始状态为 $Q_2Q_1Q_0 = 000$, 根据转换表或状态图, 可画出时序图如图 6.2.9 所示。

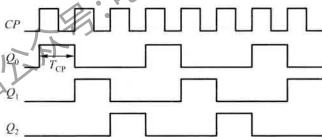


图 6.2.9 例 6.2.3 的时序图

(5) 逻辑功能分析

仅由转换表是不太容易观察该电路逻辑功能的, 而由状态图可见, 电路的有效状态是三位循环码。从时序图可以看出, 电路正常工作时, 各触发器的 Q 端轮流出现一个脉冲信号, 其宽度为一个 CP 周期, 即 $1T_{CP}$, 循环周期为 $3T_{CP}$, 这个动作可以看作是 在 CP 脉冲作用下, 电路把宽度为 $1T_{CP}$ 的脉冲依次分配给 Q_0 、 Q_1 、 Q_2 各端, 因此, 电路的功能为**脉冲分配器**或**节拍脉冲产生器**。

复习思考题

6.2.1 同步时序电路的分析过程可分为哪几个步骤?

6.2.2 在分析同步时序电路时,激励方程组、转换方程组和输出方程组是怎样导出的?

6.2.3 怎样通过转换方程组和输出方程组得到转换表和状态表?进而如何导出状态图和时序图?

6.3 同步时序逻辑电路的设计

时序电路设计又称为时序电路综合,其任务是根据给定的逻辑功能需求,选择适当的逻辑器件,设计出符合要求的时序电路。本节讨论的用触发器及门电路设计同步时序电路的方法是时序电路设计的基础,也是用 Verilog HDL 对时序电路进行描述及使用可编程逻辑器件设计时序电路的基础。了解这些设计方法,亦有助于理解已有集成时序电路产品的内部结构和工作原理。

6.3.1 设计同步时序逻辑电路的一般步骤

设计同步时序电路的一般过程如图 6.3.1 所示。



图 6.3.1 同步时序电路的设计过程

下面对设计过程中的主要步骤加以说明。

1. 由给定的逻辑功能建立原始状态图和原始状态表

通常,所要设计的时序电路的逻辑功能是通过文字、图形或波形图来描述的,首先必须把它们变换成规范的状态图和状态表。这种直接从图文描述得到的状态图和状态表分别称为**原始状态图**和**原始状态表**。这个过程是对实际问题进行分析的过程,具体做法是:

(1) 明确电路的输入条件和相应的输出要求,分别确定输入变量和输出变量的数目和名称。同步时序电路的时钟脉冲 CP (或 $\overline{CP}$) 一般不作为输入变量考虑。

(2) 找出所有可能的状态以及状态转换之间的关系和输入条件。不同的状态以字符来命名。可以假定一个初始状态,以该状态作为现态,根据输入条件确定输出及次态。以此类推,直到把每一个状态的输出和向下一个可能转换的状态全部找出后,则建立起原始状态图。

(3) 根据原始状态图建立原始状态表。

由于以后所有的设计步骤都将在原始状态图或原始状态表的基础上进行,只有在它们全面、正确反映给定设计要求的条件下,才有可能获得成功的设计结果。

2. 状态化简

原始状态图或原始状态表很可能隐含多余的状态,去除多余状态的过程称为**状态化简**,其目

的是减少电路中触发器及门电路的数量,但不能改变原始状态图或原始状态表所表达的全部逻辑功能。状态化简建立在等价状态的基础上:如果两个状态分别作为现态,其任何相同输入所建立的次态及产生的输出均完全相同,则这两个状态称为**等价状态**。凡是两个等价状态都可以合并成一个状态而不改变输入-输出关系。我们将通过 6.3.2 节的实例具体说明。

3. 状态分配

对每个状态指定一个特定的二进制代码,称为**状态分配**或**状态编码**。编码方案不同,设计出的电路结构也就不同。编码方案选择得当,设计出的电路可能工作更可靠,也可能结构相对简单。

首先,要确定状态编码的位数。同步时序电路的状态取决于触发器的状态组合,触发器的个数 N 即状态编码的位数。 N 与所要求的状态数 M 须满足如下关系

$$M \leq 2^N \quad (6.3.1)$$

其次,要对每个状态确定编码。从 2^N 个状态中取 M 个状态组合可能存在多种不同方案,随着 N 值的增大,编码方案的数目会急剧增多^①,面对大量的编码方案是难以一一进行仔细比较的。一般来说,选取的编码方案应该有利于所选触发器的激励方程及输出方程的化简以及电路的稳定可靠。有时,遵循状态变化的顺序,以二进制数自然递增顺序编码可简化电路。而使用具有一定特征的编码,如格雷码,则有利于降低输出信号产生竞争-冒险的可能性。

状态分配完成,则可将状态表中的状态名替换为状态编码,得到转换表。

4. 选择触发器类型

触发器类型选择的余地实际上是非常小的。小规模集成电路的触发器产品,大多是 D 触发器和 JK 触发器,选择具有较强逻辑功能的 JK 触发器,有时可简化激励电路。如前所述,很多可编程逻辑器件中采用 D 触发器来实现时序电路设计,如有特殊要求,用 D 触发器也非常容易构成其他逻辑功能的触发器。

5. 确定激励方程组和输出方程组

根据转换表、用卡诺图或其他方式对逻辑函数进行化简,可求得电路的激励方程组和输出方程组。这两个方程组决定了同步时序电路的组合电路部分。

6. 画出逻辑图,并检查自校正能力

按照前一步导出的激励方程组和输出方程组,可画出接近工程实现的逻辑电路图。

有些同步时序电路设计中会出现没有用到的无效状态,当电路上电后有可能陷入这些无效状态而不能退出,因此,设计的最后一步应检查电路是否能进入有效状态,即是否具有自校正能力。如果不能自校正,则需修改设计。

有时序电路要求必须从指定的初始状态开始工作,而不允许从任何其他状态启动。这时,应利用触发器的直接置 0、置 1 功能,在开始工作之前先将电路置为有效状态。图 6.3.2(a) 所示

^① 研究证明,从 N 位编码中取 M 个状态,其可能的状态分配方案数目为 $S = \frac{2^N!}{(2^N - M)!}$ 。例如, $N=3, M=5$, 其可能的编码

方案总数达 6 720 之多!

为一低电平手动复位信号产生电路,当按键开关按下,则产生低电平复位信号 $\overline{RESET}$,按键抬起,系统则进入正常工作状态。图 6.3.2(b)是一种上电自动复位电路,它利用电容两端电压不能突变的原理,在上电之初将复位信号 $\overline{RESET}$ 保持在低电平,将系统预置为初始状态。随着电源 $+V_{CC}$ 通过电阻 R 对电容 C 充电,使 C 两端电压逐渐升高,经过一段时间之后, $\overline{RESET}$ 信号才跨越逻辑阈值转变为高电平,使系统脱离复位状态而进入正常工作状态。虽然设计了复位电路的时序电路在正常工作时可不考虑自校正问题,但是,如果外界干扰等偶发因素使时序电路进入无效的循环状态,则可能出现所谓的“死机”现象,这是逻辑电路设计者不得不预先考虑的问题。

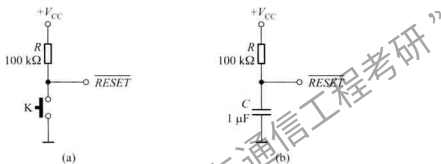


图 6.3.2 复位电路

(a) 手动复位电路 (b) 上电自动复位电路

需要说明的是,上述步骤是设计同步时序电路的一般化过程,实际设计中并不是每一步都必须执行,可根据具体情况简化或省略一些步骤,这通过下面的设计实例可以看出。

6.3.2 同步时序逻辑电路设计举例

例 6.3.1 用 D 触发器设计一个同步递增 8421 BCD 计数器。

解: 计数器实际上是对时钟脉冲进行计数,每到来一个时钟脉冲触发沿,计数器改变一次状态。在每个时钟脉冲作用下,计数器的状态输出编码值加 1,编码顺序与 8421 BCD 码一致。每经过 10 个时钟脉冲,计数器完成一个计数周期,共有 10 个状态,可用 0~9 十个数字命名各个状态。由于电路的状态数、状态转换关系及状态编码等都是明确的,因此设计过程较简单,没有必要拘泥于上一小节所述的设计步骤。

(1) 列出转换表

10 个状态的计数器共需要 4 个 D 触发器构成。由于 D 触发器功能较简单,可将激励信号同时列入转换表中,如表 6.3.1 所示。这是一个穆尔型时序电路,输出变量即状态编码。

表 6.3.1 同步 8421 BCD 计数器的转换表

Q_3^n	Q_2^n	Q_1^n	Q_0^n	$Q_3^{n+1}(D_3)$	$Q_2^{n+1}(D_2)$	$Q_1^{n+1}(D_1)$	$Q_0^{n+1}(D_0)$
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0

(2) 确定激励方程组

按表 6.3.1 可画出各触发器激励信号的卡诺图如图 6.3.3 所示。

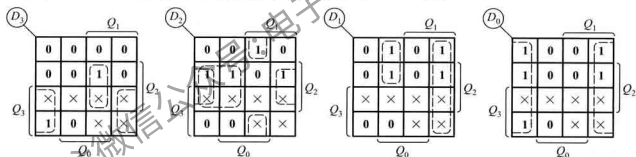


图 6.3.3 例 6.3.1 的卡诺图

4 个触发器可组合 16 个状态(0000 ~ 1111),其中有 6 个状态(1010 ~ 1111)在 8421 BCD 计数器中是无效状态,在图 6.3.3 的卡诺图中以无关项×表示。于是,得到激励方程组(在本例中也是转换方程组)

$$\begin{aligned}
 Q_3^{n+1} &= D_3 = Q_3^n \overline{Q_0^n} + Q_2^n Q_1^n Q_0^n \\
 Q_2^{n+1} &= D_2 = Q_2^n \overline{Q_1^n} + Q_2^n \overline{Q_0^n} + \overline{Q_2^n} Q_1^n Q_0^n \\
 Q_1^{n+1} &= D_1 = Q_1^n \overline{Q_0^n} + Q_3^n \overline{Q_1^n} Q_0^n \\
 Q_0^{n+1} &= D_0 = \overline{Q_0^n}
 \end{aligned}$$

(3) 画出逻辑图

根据激励方程组可画出逻辑图如图 6.3.4 所示。图中,各触发器的直接置 0 端为低电平有效,计数工作时,电路的 $\overline{RESET}$ 输入端应保持为高电平。

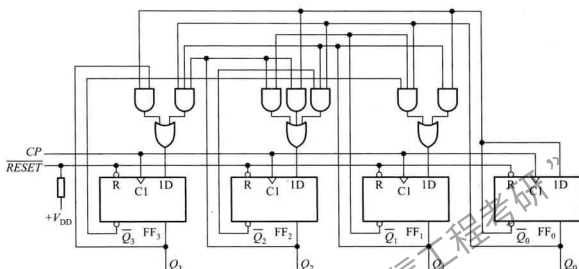


图 6.3.4 例 6.3.1 的逻辑电路

(4) 检查自校正能力

该电路有 6 个无效状态: **1010**、**1011**、**1100**、**1101**、**1110** 和 **1111**, 分别以它们作为现态, 代入电路的转换方程组而求其次态。如果还没有进入有效状态, 再以新的状态作为现态求下一个次态, 以此类推, 看最终能否进入有效状态。结果证明, 这 6 个状态在一或两个时钟周期后全部都能进入有效循环状态。

(5) 画出状态图

图 6.3.4 所示电路的完全状态图如图 6.3.5 所示。

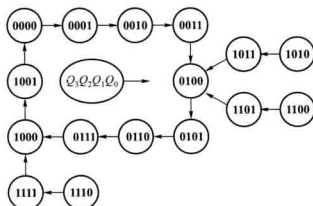


图 6.3.5 图 6.3.4 所示电路的完全状态图

如果要求电路必须从 **0000** 开始计数,则可将前述复位电路连接在 $\overline{RESET}$ 输入端。在开始计数前使 $\overline{RESET}$ 产生低电平脉冲,强制 4 个触发器进入 **0000** 的初始状态,待 $\overline{RESET}=1$ 后再开始计数。

例 6.3.2 试设计一序列编码检测器,当检测到输入信号出现 **110** 序列编码(按自左至右的顺序)时,电路输出为 **1**,否则输出为 **0**。要求用同步时序电路实现。

解: (1) 由给定的逻辑功能建立原始状态图和原始状态表

从给定的逻辑功能可知,电路有一个输入信号 A 和一个输出信号 Y ,电路功能是对输入信号 A 的编码序列进行检测,一旦检测到信号 A 出现连续编码为 **110** 序列时,输出为 **1**,检测到其他编码序列,则输出均为 **0**。

设电路的初始状态为 a ,如图 6.3.6 中大箭头所指。在此状态下,电路输出 $Y=0$,这时可能的输入有 $A=0$ 和 $A=1$ 两种情况。当 CP 脉冲相应边沿到来时,若 $A=0$,则是收到 **0**,应保持在状态 a 不变;若 $A=1$,则转向状态 b ,表示电路收到一个 **1**。当在状态 b 时,若输入 $A=0$,则表示连续输入编码为 **10**,不是 **110**,则应回到初始状态 a ,重新开始检测;若 $A=1$,则进入状态 c ,表示已连续收到

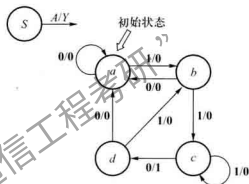


图 6.3.6 例 6.3.2 的原始状态图

两个 **1**。在状态 c 时,若 $A=0$,表明已收到序列编码 **110**,则输出 $Y=1$,并进入状态 d ;若 $A=1$,则收到编码为 **111**,应保持在状态 c 不变,看下一个编码输入是否为 $A=0$;由于尚未收到最后的 **0**,故输出仍为 **0**。在状态 d ,若输入 $A=0$,则应回到状态 a ,重新开始检测;若 $A=1$,电路应转向状态 b ,表示在收到 **110** 之后又重新收到一个 **1**,已进入下一轮检测;在 d 状态下,无论 A 为何值,输出 Y 均为 **0**。根据上述分析,可以得到如图 6.3.6 所示的原始状态图和表 6.3.2 的原始状态表。

表 6.3.2 例 6.3.2 的原始状态表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$a/0$	$c/0$
c	$d/1$	$c/0$
d	$a/0$	$b/0$

(2) 状态化简

观察表 6.3.2 中 a 和 d 两行可以看出,在 $A=0$ 和 $A=1$ 时,分别具有相同的次态 a, b 及相同的输出 **0**,因此, a 和 d 是等价状态,可以合并。这里选择去除 d 状态,并将其其他行中的次态 d 改为 a 。于是,得到化简后的状态表如表 6.3.3 所示,状态图亦可作相应化简。从实际物理意义看

也不难理解这种化简:当进入 c 状态后,电路已连续接收到两个 1 ,这时输入若为 0 ,则意味着已接收到编码 110 ,下一步电路应回到初始状态 a ,以准备新一轮检测,原始状态表中的 d 状态显然是多余的。

表 6.3.3 例 6.3.2 经化简的状态表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$a/0$	$c/0$
c	$a/1$	$c/0$

(3) 状态分配

化简后的状态有三个,可以用两位二进制代码组合 $(00, 01, 10, 11)$ 中的任意三个代码表示,用两个触发器组成电路。观察表 6.3.3,当输入信号 $A=1$ 时,有 $a \rightarrow b \rightarrow c$ 的变化顺序,当 $A=0$ 时,又存在 $c \rightarrow a$ 的变化。综合两方面考虑,这里采取 $00 \rightarrow 01 \rightarrow 11 \rightarrow 00$ 的变化顺序,可能会使其中的组合电路相对简单。于是,令 $a=00, b=01, c=11$,状态分配后得到的状态图如图 6.3.7 所示。

(4) 选择触发器类型

选用逻辑功能较强的 JK 触发器可能得到较简化的组合电路。

(5) 确定激励方程组和输出方程组

用 JK 触发器设计时序电路时,电路的激励方程需要间接导出。表 5.5.2 提供了 JK 触发器在不同现态和输入条件下所对应的次态。而在时序电路设计时,状态表已列出现态到次态的转换关系,希望推导出触发器的激励条件。所以需将特性表做适当变换,以给定的状态转换为条件,列出所需求的输入信号。这样的表格称为激励表。根据表 5.5.2 建立的 JK 触发器激励表如表 6.3.4 所示。表中的 $\times$ 表示其逻辑值与该行的状态转换无关。

表 6.3.4 JK 触发器的激励表

Q^n	Q^{n+1}	J	K
0	0	0	$\times$
0	1	1	$\times$
1	0	$\times$	1
1	1	$\times$	0

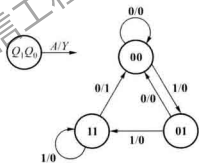


图 6.3.7 例 6.3.2 状态分配后的状态图

根据图 6.3.7 和表 6.3.4 可以列出状态转换真值表及两个触发器所要求的激励信号如表

6.3.5 所示。据此,分别画出两个触发器的输入 J 、 K 和电路输出 Y 的卡诺图,如图 6.3.8 所示。图中,不使用的状态均以无关项 $\times$ 填入。化简后得到激励方程组

$$J_1 = A Q_0 \quad K_1 = \bar{A}$$

$$J_0 = A \quad K_0 = \bar{A}$$

和输出方程

$$Y = \bar{A} Q_1$$

表 6.3.5 例 6.3.2 的状态转换真值表及激励信号

Q_1^n	Q_0^n	A	Q_1^{n+1}	Q_0^{n+1}	Y	激励信号			
						J_1	K_1	J_0	K_0
0	0	0	0	0	0	0	$\times$	0	$\times$
0	0	1	0	1	0	0	$\times$	1	$\times$
0	1	0	0	0	0	0	$\times$	$\times$	1
0	1	1	1	1	0	1	$\times$	$\times$	0
1	1	0	0	0	1	$\times$	1	$\times$	1
1	1	1	1	1	0	$\times$	0	$\times$	0

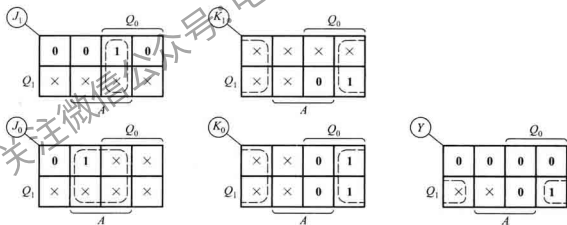


图 6.3.8 激励信号及输出信号的卡诺图

(6) 画出逻辑图

根据激励方程组和输出方程画出逻辑图,如图 6.3.9 所示。

(7) 检查自校正能力

最后还应检查该电路的自校正能力。当电路进入无效状态 **10** 后,由激励方程组和输出方程可知,若 $A=0$,则次态为 **00**;若 $A=1$,则次态为 **11**,电路能自动进入有效序列。但从输出来看,若

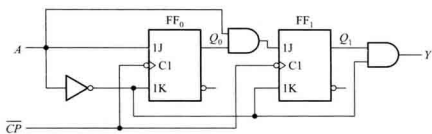


图 6.3.9 例 6.3.2 的逻辑图

电路在无效状态 **10**, 当 $A=0$ 时, 输出错误地出现 $Y=1$ 。为此, 需要对输出方程做适当修改, 即将图 6.3.8 中输出信号 Y 的卡诺图里无关项 $Q_1\bar{Q}_0\bar{A}$ 不画在包围圈内, 则输出方程变为 $Y=Q_1Q_0\bar{A}$ 。根据此式对图 6.3.9 也做相应的修改即可。

如果发现所设计的电路不能自校正, 则应修改设计。方法是: 在激励信号卡诺图的包围圈中, 对无关项 $\times$ 的处理作适当修改, 即原来取 **1** 圈入包围圈的, 可试取 **0** 而不圈入包围圈, 与上述对输出 Y 的处理方法类似。于是, 得到新的激励方程组和逻辑图, 然后再检查其自校正能力, 直到能自校正为止。

例 6.3.3 试用 D 触发器设计一同步时序电路, 实现图 6.3.10 所示原始状态图的逻辑功能。

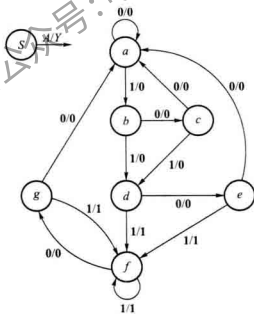


图 6.3.10 例 6.3.3 的原始状态图

解: (1) 列出原始状态表

根据图 6.3.10 列出原始状态表如表 6.3.6 所示。

表 6.3.6 例 6.3.3 的原始状态表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$c/0$	$d/0$
c	$a/0$	$d/0$
d	$e/0$	$f/1$
e	$a/0$	$f/1$
f	$g/0$	$f/1$
g	$a/0$	$f/1$

(2) 状态化简

观察表 6.3.6 发现,状态 e, g 是等价状态,可以合并。表 6.3.7 示出第一步化简的结果:将状态 g 一行删除,并用状态 e 替换表 6.3.6 中“ S^{n+1}/Y ”栏中的状态 g 。

表 6.3.7 表 6.3.6 的第一步化简

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$c/0$	$d/0$
	$a/0$	$d/0$
d	$e/0$	$f/1$
e	$a/0$	$f/1$
f	$e/0$	$f/1$

再观察表 6.3.7,又出现状态 d 和 f 是等价的,状态 f 亦可去除,代之以 d 。于是,得到表 6.3.8 所示状态表。检查该表,已不存在等价状态,因此是最简状态表。根据表 6.3.8 画出的状态图如图 6.3.11 所示。

表 6.3.8 例 6.3.3 的最简状态表

S^n	S^{n+1}/Y	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$c/0$	$d/0$
c	$a/0$	$d/0$
d	$e/0$	$d/1$
e	$a/0$	$d/1$

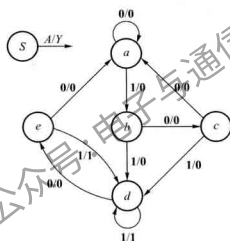


图 6.3.11 化简后的状态图

上述状态化简过程将原有的 7 个状态化简为 5 个,而输入-输出特性与原始状态表和原始状态图完全相同。

(3) 状态分配

表 6.3.8 中列出 5 个状态,最简单的状态分配是使用自然二进制码,取二进制计数序列的前 5 个连续编码,如表 6.3.9 中的状态分配方案 1。结合图 6.3.11 可以看出,这种二进制编码值的递增顺序基本上与相应的状态转换顺序一致。对于较简单的时序电路,按这种状态分配方案构成的电路,其组合电路将可能相对简单一些。

表 6.3.9 例 6.3.3 的三种状态分配方案

状态	方案 1 自然二进制码	方案 2 格雷码	方案 3 “一对一”
<i>a</i>	0 0 0	0 0 0	0 0 0 0 1
<i>b</i>	0 0 1	0 0 1	0 0 0 1 0
<i>c</i>	0 1 0	0 1 1	0 0 1 0 0
<i>d</i>	0 1 1	0 1 0	0 1 0 0 0
<i>e</i>	1 0 0	1 1 0	1 0 0 0 0

97

表 6.3.9 中所列方案 2 为格雷码方案。如果状态图所示出的状态转换顺序是简单的从 *a* 到 *e*, 那么它从一个状态转换到下一状态仅有一个触发器改变状态, 使用这种方案将降低电路发生竞争-冒险的可能, 提高电路的可靠性。但遗憾的是, 本例的状态转换并不是简单的单循环序列。

表 6.3.9 中的方案 3 为“一对一”编码方案^①。虽然这将使用较多的触发器, 但它的编码方式非常简单, 可有效地简化组合电路, 并换得工作可靠性和工作速度的提高。在大规模可编程逻辑器件, 如 FPGA 中, 触发器数量较多而门逻辑相对较少, “一对一”的编码方案有时反而更有利于提高器件资源的利用率。

本例按表 6.3.9 中方案 1 分配状态, 得到的转换表如表 6.3.10 所示。

表 6.3.10 例 6.3.3 的转换表

$Q_2^* Q_1^* Q_0^*$	$Q_2^{*+1} Q_1^{*+1} Q_0^{*+1} / Y$	
	<i>A</i> = 0	<i>A</i> = 1
0 0 0	0 0 0 / 0	0 0 1 / 0
0 0 1	0 1 0 / 0	0 1 1 / 0
0 1 0	0 0 0 / 0	0 1 1 / 0
0 1 1	1 0 0 / 0	0 1 1 / 1
1 0 0	0 0 0 / 0	0 1 1 / 1

(4) 确定激励方程组和输出方程组

该电路需要用 3 个 *D* 触发器实现, 它们的输出分别为 Q_2 、 Q_1 、 Q_0 。于是, 由表 6.3.10 可得状态转换真值表如表 6.3.11 所示。

由于 *D* 触发器的特性方程为 $Q^{*+1} = D$, 所以根据表 6.3.11, 可以画出 D_2 、 D_1 、 D_0 和 *Y* 的卡诺图如图 6.3.12 所示, 未应用的状态可作为无关项 $\times$ 填入。

^① 有些国外文献称之为“One-Hot Encoding”, 国内某些文献直译为“1 位热码编码”。

表 6.3.11 例 6.3.3 的状态转换真值表

Q_2^n	Q_1^n	Q_0^n	A	$Q_2^{n+1}(D_2)$	$Q_1^{n+1}(D_1)$	$Q_0^{n+1}(D_0)$	Y
0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0
0	0	1	0	0	1	0	0
0	0	1	1	0	1	1	0
0	1	0	0	0	0	0	0
0	1	0	1	0	1	1	0
0	1	1	0	1	0	0	0
0	1	1	1	0	1	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	1	1	1

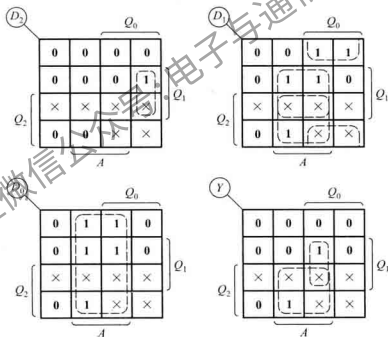


图 6.3.12 表 6.3.11 的卡诺图

经卡诺图化简,得到三个触发器的激励方程组

$$D_2 = Q_1^n Q_0^n \bar{A}$$

$$D_1 = \overline{Q_1}^n Q_0^n + Q_1^n A + Q_2^n A$$

$$D_0 = A$$

和输出方程

$$Y = Q_1 Q_0 A + Q_2 A$$

经过几次实践之后,实际上可不必列状态转换真值表而直接从转换表得到卡诺图。

(5) 画出逻辑图

根据激励方程组和输出方程,可以画出图 6.3.13 所示的逻辑图。电路中引出了直接复位端 $\overline{RESET}$,可外接复位电路,强制电路在开启上电时从状态 **000** 开始工作。

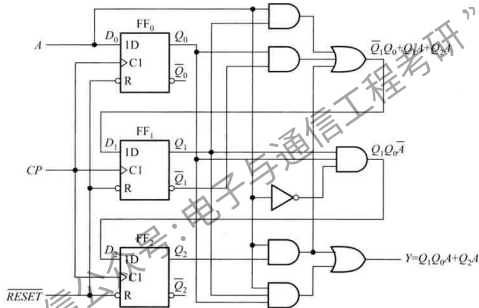


图 6.3.13 例 6.3.3 的逻辑图

(6) 检查自校正能力

该电路有 3 个无效状态: **101**、**110** 和 **111**, 需要评估电路受到外界干扰时万一落入这 3 个无效状态时的影响。将 **101**、**110** 和 **111** 分别作为现态, 与可能出现的所有输入信号一起分别代入电路的转换方程组而求其次态。结果证明, 无论在何种情况下, 这 3 个状态在一个时钟周期后全部都能进入有效状态。于是, 可进一步画出完全状态图如图 6.3.14 所示。

6.3.3 同步时序逻辑电路中的时钟偏移

一般的时序电路都含有多个触发器, 特别是规模较大的时序电路(如 PLD 中), 其触发器的数量将会更多。同步时序电路要求时钟脉冲必须同时连接到每个触发器的时钟输入端。当电路规模较大时, 由于实际电路制版时触发器的分布位置和布线等限制, 每个触发器的时钟信号线的长度可能存在较大差异。另外, 当触发器数量较多时, 由于公共时钟信号驱动电路的负载能力有

限(即扇出的限制),经常需要在某些位置添加缓冲器来提高时钟信号线的驱动能力,由此造成时钟脉冲传输到不同触发器的延迟时间不同。这种从同一时钟源发出的时钟脉冲,通过不同路径到达每个触发器的时间不同而产生的偏差称为**时钟偏移**。随着工艺水平的提高,触发器的传输延迟时间大大缩短(为 $1 \sim 2 \text{ ns}$, 参见表 5.4.2), 很有可能出现小于时钟偏移的情况, 此时, 将导致同步时序电路出现错误动作。

例如某同步时序逻辑电路中的 2 个触发器电路如图 6.3.15(a) 所示, 触发器 FF_0 的输出 Q_0 作为触发器 FF_1 的输入, 且假设它们的连线很短, 可以忽略其延迟。另外, 假设受元器件位置及布线等的限制, 由 CP 到 FF_1 的时钟 CP_1 的传输路径很长, 有较大延迟, 即时钟偏移较大。在不考虑时钟偏移时, 电路按照设计初衷正常工作, 第 1 个 CP 脉冲有效沿到来时, 输入信号 A 的状态存入 FF_0 。第 2 个 CP 脉冲到来时, A 的状态才传递到 FF_1 , 其工作波形应如图 6.3.15(b) 所示(设触发器初态 $Q_0, Q_1 = 00$)。但是, 实际电路存在时钟偏移, 如果时钟偏移量超过触发器 FF_0 的传输延迟时间 t_{pd} , 即 FF_1 的时钟 CP_1 在 Q_0 翻转后才到

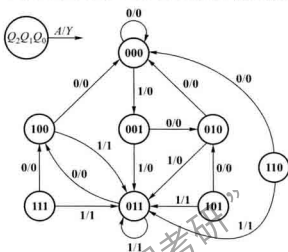


图 6.3.14 例 6.3.3 的完全状态图

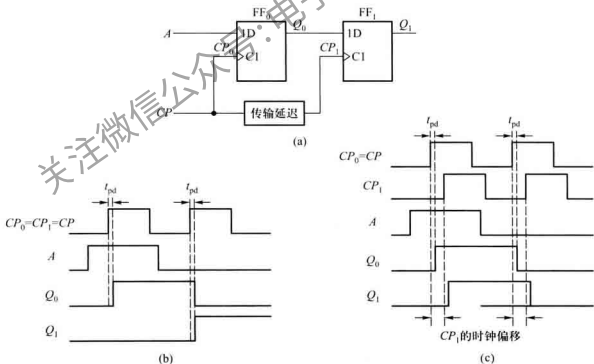


图 6.3.15 某同步时序逻辑电路工作情况

(a) 逻辑电路 (b) 无时钟偏移时的时序图 (c) 由时钟偏移引起的错误时序图

达,如图 6.3.15(c)所示,则 Q_1 的波形将与正常工作时(图 6.3.15(b))不同,即 A 的状态在第 1 个 CP 脉冲到来时就已经传递到了 FF₁,出现了逻辑错误。

由此看出,严重的时钟偏移很有可能造成同步时序电路的工作错误。为了消除 6.3.15(a)中时钟偏移带来的影响,一方面尽可能缩短 CP₁ 的传输路径,减小其延迟;另一方面,也可以在 CP₀ 的传输路径上添加缓冲器,延长其延迟,从而减小两触发器的时钟偏移。

另外,不同的时钟传输路径上的负载(特别是容性负载)不平衡时,将导致不同路径上时钟脉冲的上升沿和下降沿的斜率不同,也会引起时钟偏移。因此,设计时钟布线网络时,还需要考虑各传输路径上负载的平衡问题。

通过以上分析,引起时钟偏移的主要原因可归纳为:

- (1) 各触发器时钟传输路径的长度不同;
- (2) 各触发器时钟传输路径上经过的缓冲器的数量不同;
- (3) 各触发器时钟传输路径上的负载不平衡。

在设计大规模的同步时序逻辑电路时,为了减少时钟偏移对电路正常工作的影响,应尽可能减小各触发器的时钟偏移。图 6.3.16 是一种可以有效减小时钟偏移的 H 型时钟布线网络。该方案的设计思路是使同步时钟信号传输到每个触发器 CP 端的路径尽可能相同。为简洁起见,图中未画出完整的触发器,而仅以 FF_i 替代。考虑到系统时钟 CP 扇出数的限制,网络中添加了 4 个缓冲器,每个缓冲器驱动 8 个触发器负载。当然,在实际电路设计时,可根据触发器 CP 端的负载效应和缓冲器的驱动能力来安排缓冲器的位置和数量。

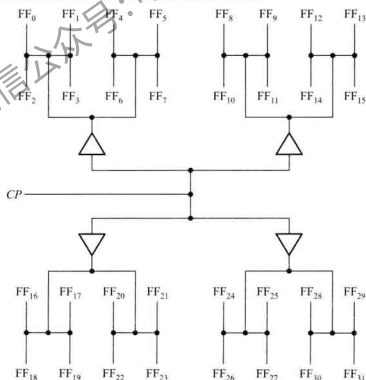


图 6.3.16 H 型时钟布线网络

复习思考题

- 6.3.1 同步时序电路的设计过程可分为哪几个步骤?
- 6.3.2 什么是原始状态图和原始状态表? 怎样建立原始状态图和原始状态表?
- 6.3.3 什么是等价状态? 若状态 a 与 b 等价, b 又与 c 等价, 那么状态 a 与 c 等价吗?
- 6.3.4 同步时序电路中触发器的数目与状态数目有何关系? 与状态分配又有何关系?
- 6.3.5 激励方程组和输出方程组是如何确定的? 它们可决定同步时序电路中哪些电路?
- 6.3.6 如何检查同步时序电路的自校正能力? 如果要求电路从特定状态开始工作, 应如何处理?
- 6.3.7 造成时钟偏移的原因是什么? 一般如何解决?

6.4 异步时序逻辑电路的分析

本节主要讨论用触发器构成的脉冲异步时序电路的分析方法。异步时序电路与同步时序电路的主要区别在于电路中没有统一的时钟脉冲, 因而各存储电路不是同时更新状态, 状态之间没有准确的时间分界。在分析脉冲异步时序电路时必须注意以下几点。

(1) 分析状态转换时必须考虑各触发器的时钟信号作用情况

异步时序电路中, 由于各个触发器只有在其 CP_n (或 $\overline{CP}_n$, 下标 n 表示电路中第 n 个触发器) 端的信号有效时, 才有可能改变状态。因此, 在分析状态转换时, 首先应根据给定的电路列出各个触发器的时钟信号方程组, 根据该方程组的各表达式分别确定相应触发器的 CP_n (或 $\overline{CP}_n$) 端是否有时钟信号的作用, 并用变量 cp_n 表示: 有作用, 则令 $cp_n = 1$; 否则 $cp_n = 0$ 。对于上升沿触发的触发器, 当其 CP_n 端的信号由 0 变 1 时 $cp_n = 1$; 反之, 对下降沿触发的触发器, 则在 $\overline{CP}_n$ 信号由 1 变 0 时 $cp_n = 1$ 。然后再根据激励信号确定那些 $cp_n = 1$ 的触发器的次态, $cp_n = 0$ 的触发器则保持原有状态不变。这里, cp_n 不是一个逻辑变量, 但参与各转换方程的逻辑运算。

(2) 每一次状态转换必须从输入信号所能触发的第一个触发器开始逐级确定

同步时序电路的分析可以从任意一个触发器开始推导状态的转换, 而异步时序电路每一次状态转换的分析必须从输入信号所能作用的第一个触发器开始推导, 确定它的状态变化, 然后根据它的输出信号分析下一个触发器的时钟信号以确定 cp_n 的值, 进一步决定该触发器是否发生状态转换。像这样依次逐级分析, 直到最后一个触发器。待全部触发器的转换状态导出后, 才能最终确定电路的次态, 填入转换表或状态图。

(3) 每一次状态转换都有一定的时间延迟

同步时序电路的所有触发器几乎是同时更新状态的, 与之不同, 异步时序电路各个触发器之间的状态更新存在一定的延迟, 也就是说, 从现态 S^n 到次态 S^{n+1} 的转换过程中有一段各触发器分别延迟更新状态的“不稳定”时间。在此期间, 整个电路的状态是不确定的。只有当全部触发

器的状态更新完毕,电路才进入新的“稳定”状态,即次态 S^{n+1} 。因此,异步时序电路的输入信号(包括时钟信号)必须等待电路进入稳定状态之后才允许发生改变,否则电路会处在不确定的状态。由于上述延迟时间的存在,对于同一系列的逻辑电路,类似功能的同步时序电路的速度要快于异步时序电路。

下面以两个实例来说明异步时序电路的分析过程。

例 6.4.1 分析图 6.4.1 所示逻辑电路。

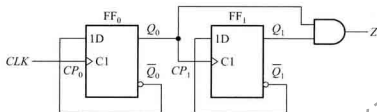


图 6.4.1 例 6.4.1 的逻辑电路图

解：在图 6.4.1 的电路中,两触发器 FF_0 和 FF_1 的 CP_0 和 CP_1 未共用时钟信号,故属于异步时序电路。

(1) 列出各逻辑方程组

① 时钟信号方程组

根据逻辑图列出各触发器的时钟信号表达式

$$CP_0 = CLK$$

$$CP_1 = Q_0$$

② 激励方程组

$$D_0 = \overline{Q_0}$$

$$D_1 = \overline{Q_1}$$

③ 转换方程组

这时需要考虑各触发器时钟信号 CP_n 的作用:只有 $cp_n = 1$ 发生后,相应触发器才可能转换状态,当 $cp_n = 0$,即 $\overline{cp_n} = 1$ 时,触发器应保持原态。因此,触发器的特性方程中应引入 cp_n 而改写为如下的转换方程组:

$$Q_0^{n+1} = D_0 cp_0 + Q_0^n \overline{cp_0} = \overline{Q_0^n} cp_0 + Q_0^n \overline{cp_0} \quad (6.4.1)$$

$$Q_1^{n+1} = D_1 cp_1 + Q_1^n \overline{cp_1} = \overline{Q_1^n} cp_1 + Q_1^n \overline{cp_1} \quad (6.4.2)$$

式(6.4.1)和式(6.4.2)中右边的第二项表示,当触发器没有时钟信号作用时,状态保持不变。

④ 输出方程组

$$Z = Q_1 Q_0$$

(2) 列出转换表

列转换表的方法与同步时序电路分析过程基本相似,只是还应注意各触发器是否存在 $cp_n = 1$,因此,可在转换表中增加 cp_0 、 cp_1 两列。对应于输入信号 CLK 的每一个上升沿, $cp_0 = 1$,将 cp_0 和 FF_0 的现态 Q_0^n 代入式(6.4.1),从而得到 Q_0^{n+1} 。因为只有对应于 Q_0 由 0 到 1 的跳变,才有 $cp_1 = 1$,而对应于 Q_0 由 1 到 0 的转换, $cp_1 = 0$,所以在每一次 Q_0 的状态转换后,应首先根据 FF_1 时钟信号的逻辑表达式确定 cp_1 ,然后将 FF_1 的现态 Q_1^n 和相应的 cp_1 代入式(6.4.2),求得其次态 Q_1^{n+1} 。如果从 $Q_1^n Q_0^n$ 为 00 开始推导,可以得到如表 6.4.1 所示的转换表。

表 6.4.1 例 6.4.1 的转换表

Q_1^n	Q_0^n	cp_1	cp_0	Q_1^{n+1}	Q_0^{n+1}	Z
0	0	1	1	1	1	0
0	1	0	1	0	0	0
1	0	1	1	0	1	0
1	1	0	1	1	0	1

(3) 画出状态图和时序图

由表 6.4.1 所示的转换表可画出如图 6.4.2 所示的状态图。

根据状态图和具体触发器的传输延迟时间 t_{pd} 和 t_{fall} ,可以画出时序图,如图 6.4.3 所示。可以看出,由于两个触发器异步翻转之间存在延迟,电路有短时间存在着不确定的状态,如果使用 74HCT74 双 D 触发器实现图 6.4.1 所示的电路,这段时间大约在 40 ns 左右。

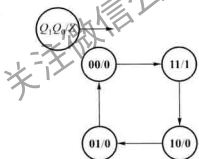


图 6.4.2 例 6.4.1 的状态图

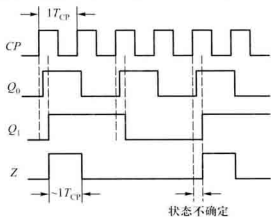


图 6.4.3 例 6.4.1 的时序图

(4) 逻辑功能分析

由状态图和时序图可知,该电路是一个异步二进制递减计数器,Z 信号的上升沿可触发借位操作。也可把它看作是一个序列信号发生器,输出序列脉冲信号 Z 的重复周期为 $4T_{CP}$,脉宽约为 $1T_{CP}$ 。

例 6.4.2 分析图 6.4.4 所示逻辑电路。

$cp_1=0$, Q_1 也将保持原状。 CLK 信号第一个上升沿到来后, 电路状态改变为 **001**。以此类推, 可得电路的转换表, 如表 6.4.2 所示。

表 6.4.2 例 6.4.2 的转换表

Q_2^n	Q_1^n	Q_0^n	cp_2	cp_1	cp_0	Q_2^{n+1}	Q_1^{n+1}	Q_0^{n+1}
0	0	0	0	0	1	0	0	1
0	0	1	0	1	1	0	1	0
0	1	0	0	0	1	0	1	1
0	1	1	1	1	1	1	0	0
1	0	0	1	0	0	0	0	0
1	0	1	1	0	0	0	0	1
1	1	0	1	0	0	0	1	0
1	1	1	1	0	0	0	1	1

(3) 画出状态图

由表 6.4.2 所示的转换表可画出状态图, 如图 6.4.5 所示。该图表明, 当电路处于循环外的状态(**101**、**110** 和 **111** 三个状态)时, 在 CLK 信号出现第一个上升沿后, 电路便能进入有效循环状态。

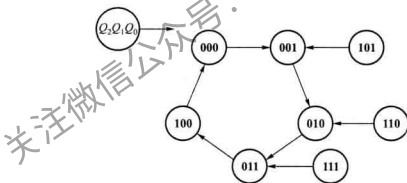


图 6.4.5 例 6.4.2 的状态图

(4) 逻辑功能分析

由状态图可知, 图 6.4.4 所示电路是一个具有 **000 ~ 100** 五个连续二进制编码状态的异步计数器。

复习思考题

- 6.4.1 异步时序电路的分析与同步时序电路的分析主要有哪些不同之处?
- 6.4.2 分析异步时序电路时,为什么需要列出触发器的时钟信号方程组?
- 6.4.3 为什么在分析异步时序电路的状态转换时,必须按信号作用的顺序对每个触发器进行逐个推导,才能确定电路的状态?
- 6.4.4 异步时序电路在每次状态转换时为什么会出现短时间的不稳定状态?这种不稳定的延续时间与门电路和触发器的延迟时间有什么关系?此外还可能与哪些因素相关?
- 6.4.5 为什么异步时序电路的输入信号必须在电路转换状态稳定之后才允许发生改变?

6.5 若干典型的时序逻辑电路

本节介绍在数字系统中广泛应用的几种典型时序电路——寄存器、移位寄存器和计数器的电路结构和工作原理,它们是时序电路的基本逻辑部件,与各种组合电路一起,可以构成逻辑功能极其复杂的数字系统。寄存器、移位寄存器和计数器有很多种类的中规模集成电路,可直接用于较简单的数字系统。对于较复杂的时序电路设计,应选择可编程逻辑器件实现,大批量生产的电子产品采用专用集成电路则可降低成本。本节提及的几种中规模集成电路,都经过精心设计和多年改进,具有较完善的逻辑功能及电路性能,至今仍广泛应用。它们的设计思想和电路结构,可作为可编程逻辑器件或专用集成电路设计的范例。在一些逻辑集成开发软件中也包含这些电路的“宏模块”,直接引用可简化数字系统设计。总之,充分理解这些典型时序电路的工作原理和电路结构并灵活应用,对于各种数字系统的设计都有裨益。

6.5.1 寄存器和移位寄存器

1. 寄存器

寄存器是数字系统中用来存储二进制数据的逻辑部件。1个触发器可存储1位二进制数据,存储 N 位二进制数据的寄存器需要用 N 个触发器组成。

由8个触发器构成的8位寄存器的逻辑图如图6.5.1所示。图中, $D_7 \sim D_0$ 是8位数据输入端,在 CP 脉冲上升沿作用下, $D_7 \sim D_0$ 端的数据同时存入相应的触发器,是一种极为简单的同步时序电路。当输出使能控制信号 $\overline{OE}=0$ 时,触发器存储的数据通过三态门输出端 $Q_7 \sim Q_0$ 并行输出。表6.5.1是典型的中规模集成8位寄存器74HC/HCT374的功能表。

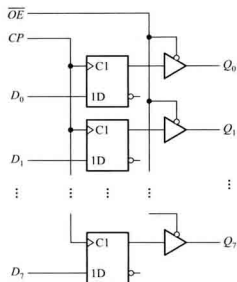


图 6.5.1 带输出缓冲器的 8 位寄存器

表 6.5.1 74HC/HCT374 的功能表

工作模式	输 入			内部触发器	输出
	$\overline{OE}$	CP	D_n	Q_n^{n+1}	$Q_0 \sim Q_7$
存入和读出数据	L	$\uparrow$	L^*	L	相应内部触发器的状态
	L		H^*	H	
存入数据, 禁止输出	H	$\uparrow$	L^*	L	高阻
	H	$\uparrow$	H^*	H	高阻

注: D_n 和 Q_n^{n+1} 的下标表示第 n 位触发器。 L^* 和 H^* 表示 CP 脉冲上升沿之前瞬间 D_n 的电平。

从存储数据的角度看, 5.3.2 小节介绍的 8 D 锁存器 74HC/HCT373 与此处的 8 位寄存器 74HC/HCT374 具有类似的逻辑功能。两者的区别在于, 前者是电平敏感电路, 而后者是脉冲边沿敏感电路。它们有不同的应用场合, 主要取决于控制信号与数据信号之间的定时关系, 以及控制存储数据的方式。如果数据状态的更新可能出现在控制信号(锁存器的 LE)开始有效之后, 则只能使用锁存器, 它不能保证各个锁存器输出同时更新状态; 如果能确保输入数据的更新在控制信号(寄存器的 CP)触发边沿出现之前稳定, 并要求输出同时更新状态, 则可选择寄存器。一般来说, 寄存器比锁存器具有更好的同步性能和抗干扰性能。

与“宽总线”锁存器类似, 寄存器也有 16 位、18 位、20 位和 32 位的“宽总线”集成电路。为了应用的灵活方便, 一般都把其中的 D 触发器分为若干组, 各组间既可同步工作, 又可异步工作。另外, 也有 1~2 个触发器的“小逻辑”寄存器产品。

2. 移位寄存器

(1) 基本的移位寄存器

如果将若干个触发器级联成如图 6.5.2 所示电路,则构成基本的移位寄存器。图中是一个 4 位移位寄存器,串行二进制数据从输入端 D_{SI} 输入,左边触发器的输出作为右邻触发器的数据输入。若将串行数码 $D_3D_2D_1D_0$ 从高位(D_3)至低位(D_0)按时钟脉冲间隔依次送到 D_{SI} 端,经过第一个时钟脉冲后, $Q_0 = D_3$ 。由于跟随 D_3 后面的是 D_2 ,因此经过第二个时钟脉冲后,触发器 FF_0 的状态移入触发器 FF_1 ,而 FF_0 转变为新的状态,即 $Q_1 = D_3, Q_0 = D_2$ 。以此类推,可得该移位寄存器的功能如表 6.5.2 所示($\times$ 表示不确定状态)。由表可知,输入数码依次由左侧触发器移到右侧触发器。经过 4 个时钟脉冲后,4 个触发器的输出状态 $Q_3Q_2Q_1Q_0$ 与输入数码 $D_3D_2D_1D_0$ 相对应。为了加深理解,在图 6.5.3 中画出了数码 **1101**(即 $D_3 = 1, D_2 = 1, D_1 = 0, D_0 = 1$) 在移位寄存器中移位的波形,经过 4 个时钟脉冲后, **1101** 出现在触发器的输出端 $Q_3Q_2Q_1Q_0$ 。这样,就将串行输入数据转换为并行输出数据 D_{PO} 。

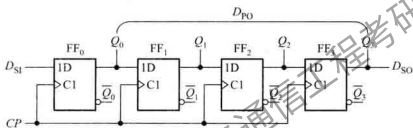


图 6.5.2 用 D 触发器构成的 4 位移位寄存器

表 6.5.2 图 6.5.2 电路的功能

CP	Q_0	Q_1	Q_2	Q_3
第一个 CP 脉冲之前	$\times$	$\times$	$\times$	$\times$
1	D_3	$\times$	$\times$	$\times$
2	D_2	D_3	$\times$	$\times$
3	D_1	D_2	D_3	$\times$
4	D_0	D_1	D_2	D_3

在图 6.5.3 中还画出了第 5 到第 8 个时钟脉冲作用下,输入数码在寄存器中移位的波形。由图可见,在第 8 个时钟脉冲作用后,数码已从串行数据输出端 D_{SO} (即 Q_3 端)全部移出寄存器。也就是说,随着时钟信号的推移,从输出端 D_{SO} 可得到 **1101** 的串行输出。

一般来说, N 位移位寄存器要由 N 个触发器构成,需要 $N \cdot T_{CP}$ 来完成串行到并行的数据转换,同样也需要 $N \cdot T_{CP}$ 来实现并行到串行的数据输出。这里, T_{CP} 为时钟周期。

从上述操作可以看出,移位寄存器只能用脉冲边沿敏感的触发器,而不能用电平敏感的锁存器来构成,因为在时钟脉冲高电平期间,锁存器输出跟随输入变化的特性将使移位操作失去控制。显然,移位寄存器属于同步时序电路。

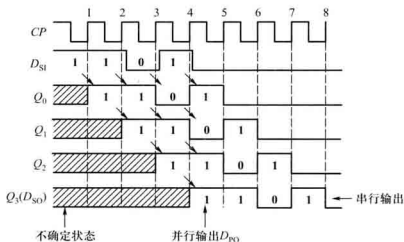


图 6.5.3 图 6.5.2 电路的时序图

(2) 多功能双向移位寄存器

① 工作原理

有时需要对移位寄存器的数据流向加以控制,实现数据的双向移动,其中一个方向称为右移,另一个方向则为左移,这种移位寄存器称为**双向移位寄存器**。由于国家标准规定,逻辑图中最低有效位(LSB)到最高有效位(MSB)的电路排列顺序应从上到下,从左到右。因此,定义移位寄存器中的数据从低位触发器移向高位为右移,反之则为左移。这一点与通常计算机程序中的规定相反,后者从自然二进制数的排列考虑,将数据移向高位定义为左移,反之则为右移。

为了扩展逻辑功能和增加使用的灵活性,在双向移位基础上,还可以增加并行输入、并行输出等功能,构成多功能移位寄存器,其工作模式的简化示意图如图 6.5.4 所示。

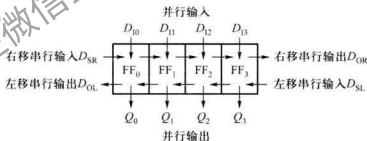


图 6.5.4 多功能移位寄存器工作模式简图

图 6.5.5 是实现数据保持、右移、左移、并行置入和并行输出的一种电路方案。图中的 D 触发器 FF_m 是 N 位移位寄存器中的第 m 位触发器,在其数据输入端插入了一个 4 选 1 数据选择器,用两位编码选择输入信号 $S_1 S_0$ 控制触发器输入信号 D_m 的来源。当 $S_1 = S_0 = 0$ 时,选择该触发器本身的输出 Q_m ,次态为 $Q_m^{n+1} = D_m = Q_m^n$,触发器保持状态不变;当 $S_1 = 0, S_0 = 1$ 时,触发器 FF_{m-1} 的输出 Q_{m-1} 被选中,故 CP 脉冲上升沿到来时, FF_m 存入 FF_{m-1} 此前的逻辑值,即 $Q_m^{n+1} = Q_{m-1}^n$,

而 $Q_{m+1}^{n+1} = Q_m^n$, 从而实现右移功能; 类似地, 当 $S_1 = 1, S_0 = 0$ 时, 数据选择器选择 Q_{m+1} , 实现左移功能; 而当 $S_1 = S_0 = 1$ 时, 则选择并行输入数据 D_{1m} , 其次态 $Q_m^{n+1} = D_{1m}$, 从而完成并行数据的置入。上述四种功能概述于表 6.5.3 中。此外, 在各触发器的输出端 $Q_{N-1} \sim Q_0$ 可以得到 N 位并行数据的输出。

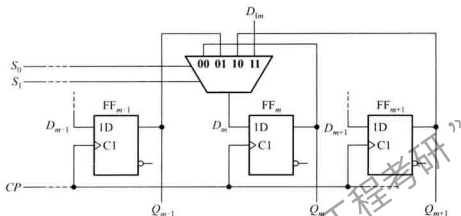


图 6.5.5 实现多种功能双向移位寄存器的一种方案

表 6.5.3 图 6.5.5 的功能表

控制信号		功 能
S_1	S_0	
0	0	保 持
0	1	右 移
1	0	左 移
1	1	并行输入

② 典型的多功能 4 位双向移位寄存器

采用图 6.5.5 方案实现数据保持、右移、左移、并行输入和并行输出的一种多功能典型的 4 位双向移位寄存器如图 6.5.6 所示。它采用了 4 个 SR 触发器分别配以非门构成 4 个 D 触发器, 连接在各触发器 1R 端上的与或非门则实现了数据选择器的功能。设连接在触发器 1R 端的与或非门输出为 $\bar{D}$, 1S 端的输入则为 D , 将二者分别代入 SR 触发器的特性方程, 得

$$Q^{n+1} = S + \bar{R}Q^n = D + \bar{D}Q^n = D$$

于是将 SR 触发器转换为 D 触发器, 功能与图 6.5.5 完全一致。图中, D_{SR} 是右移串行数据输入端, D_{SL} 是左移串行数据输入端, $D_{10} \sim D_{13}$ 是并行数据输入端, $\overline{CR}$ 连接到每个触发器的直接置 0 端, 可控制寄存器异步清零。图中输入缓冲器可以减轻前级的负载, 输出缓冲器可以提高驱动能力。图 6.5.6 所示实际是 74HC/HCT194 的内部逻辑图, 表 6.5.4 是它的功能表。表中内容: 第

1 行表示寄存器异步清零操作,第 2 行为保持状态,第 3、4 行为串行数据右移操作,第 5、6 行为串行数据左移操作,第 7 行是并行数据的同步置入操作。

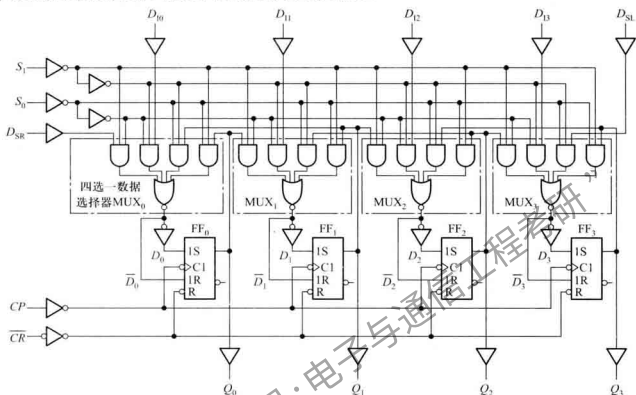


图 6.5.6 多功能 4 位双向移位寄存器 74HC/HCT194 的逻辑图

表 6.5.4 多功能 4 位双向移位寄存器 74HC/HCT194 的功能表

输 入										输 出				功 能
清零	控制信号		时钟	串行输入		并行输入				Q_0^{n+1}	Q_1^{n+1}	Q_2^{n+1}	Q_3^{n+1}	
$\overline{CR}$	S_1	S_0	CP	右移 D_{sh}	左移 D_{sl}	D_0	D_1	D_{12}	D_{13}					
L	x	x	x	x	x	x	x	x	x	L	L	L	L	异步清零
H	L	L	x	x	x	x	x	x	x	Q_0^n	Q_1^n	Q_2^n	Q_3^n	保持
H	L	H	$\uparrow$	L	x	x	x	x	x	L	Q_0^n	Q_1^n	Q_2^n	右移
H	L	H	$\uparrow$	H	x	x	x	x	x	H	Q_0^n	Q_1^n	Q_2^n	右移
H	H	L	$\uparrow$	x	L	x	x	x	x	Q_1^n	Q_2^n	Q_3^n	L	左移
H	H	L	$\uparrow$	x	H	x	x	x	x	Q_1^n	Q_2^n	Q_3^n	H	左移
H	H	H	$\uparrow$	x	x	D_{10}^n	D_{11}^n	D_{12}^n	D_{13}^n	D_{10}^n	D_{11}^n	D_{12}^n	D_{13}^n	同步并行置数

注: D_{13}^n 表示 CP 脉冲上升沿之前瞬间 D_{13} 的电平。

有时要求在移位过程中,数据仍保持在寄存器中不丢失。此时,只要将移位寄存器最高位的输出接至最低位的输入,或将最低位的输出接至最高位的输入,便可实现这个功能,称为**环形移位寄存器**。它亦可作计数器使用,称为**环形计数器**,将在 6.5.2 节中讨论。

6.5.2 计数器

计数器是最常用的时序电路之一,它们不仅可用于对脉冲进行计数,还可用于分频、定时、产生节拍脉冲以及其他时序信号。计数器的种类不胜枚举,按触发器动作分类,可分为同步计数器和异步计数器;按编码数值增减分类,可分为递增计数器、递减计数器和可逆计数器;按编码分类,又可分为二进制码计数器、BCD 计数器、循环码计数器等。计数器运行时,从某一状态开始依次遍历不重复的各个状态后完成一次循环,所经过的状态总数称为计数器的**模 (Modulo)**,并用 M 表示。若某个计数器在 n 个状态下循环计数,通常则称之为**模 n 计数器**,或 $M=n$ 计数器。例如一个在 60 个不同状态中循环转换的计数器,就可称为**模 60**,或 $M=60$ 计数器。有时也把模 n 计数器称为 n 进制计数器。

1. N 位二进制计数器

按照二进制数自然递增或递减编码的计数器称为**二进制计数器 (Binary Counter)**, N 位二进制计数器的模为 2^N ,由 N 个触发器组成。

(1) N 位异步二进制计数器

图 6.5.7 是一个 4 位异步二进制计数器的逻辑图,它由 4 个下降沿触发的 T' 触发器组成。计数脉冲 $\overline{CP}$ 加至触发器 FF_0 的时钟脉冲输入端,每输入一个计数脉冲, FF_0 翻转一次。 FF_1 、 FF_2 和 FF_3 都以前级触发器的 Q 端输出作为触发信号,当 Q_0 由 1 变 0 时, FF_1 翻转,其余类推。分析其工作过程,不难得到输出波形如图 6.5.8 所示。由图可见,从初态 **0000** (可由 $\overline{CR}$ 输入低电平脉冲使 4 个触发器全部置 0) 开始,每输入一个计数脉冲,计数器的状态就按二进制编码值递增 1,输入第 16 个计数脉冲后,计数器又回到 **0000** 状态。显然,该计数器以 16 个 $\overline{CP}$ 脉冲构成一个计数周期,是模 16 ($M=16$) 递增计数器。其中, Q_0 的频率是 $\overline{CP}$ 的 1/2,即实现了 2 分频, Q_1 得到 $\overline{CP}$ 的 4 分频,以此类推, Q_2 、 Q_3 分别对 $\overline{CP}$ 进行了 8 分频和 16 分频,因而,计数器也可作为**分频器**使用。

异步二进制计数器的工作原理和结构均较简单,但各触发器不是在同一时钟沿作用下同时翻转,而是逐级脉动翻转实现计数进位的,故亦称之为**纹波计数器 (Ripple Counter)**。图 6.5.8 中的虚线是考虑了触发器逐级翻转过程中平均传输延迟时间 t_{pd} 的波形。由于各触发器不在同一时间翻转,因此,若用这种计数器驱动组合逻辑电路,则可能出现瞬间错误的逻辑输出。例如,当计数值从 **0111** 加 1 时, Q_3 、 Q_2 、 Q_1 、 Q_0 要先后经过 **0110**、**0100**、**0000** 几个状态,才最终翻转为 **1000**。

另外,当计数脉冲频率很高时, $Q_3 \sim Q_0$ 甚至会出现编码输出分辨不清的情况。对于一个 N 位二进制异步计数器来说,从一个计数脉冲作用在第一个触发器开始,到第 N 个触发器翻转达到稳定状态为止,需要经历的时间为 Nt_{pd} 。为了保证正确地检出计数器的输出状态,必须满足

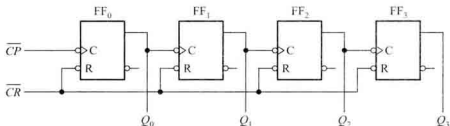


图 6.5.7 4 位异步二进制计数器逻辑图

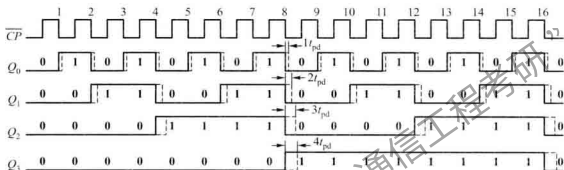


图 6.5.8 4 位异步二进制计数器时序图

$T_{cr} \gg Nt_{pd}$ 的条件, 其中, T_{cr} 为计数脉冲 CP 的周期。

(2) N 位同步二进制计数器

为了提高计数速度, 可采用同步计数器。其特点是, 计数脉冲作为时钟信号同时接于所有触发器的时钟脉冲输入端。在每次计数脉冲沿到来之前, 根据当前计数器状态, 利用组合逻辑电路产生触发器的激励条件。当计数脉冲沿到来时, 所有应翻转的触发器同步翻转, 同时, 所有应保持原状态的触发器不发生变化。由于不存在异步计数器那种纹波进位造成的延迟时间积累, 所以能取得较高的计数速度, 输出编码也不会发生纹波进位时的那种混乱。显然, 同步计数器是一种同步时序电路。

① 工作原理

表 6.5.5 是 4 位二进制计数器的转换表^①。观察该表递增计数一栏可以看出, Q_0 在每个计数脉冲作用后都要翻转一次; Q_1 只在 $Q_0=1$ 的条件下接受计数脉冲作用而翻转; Q_2 则在 $Q_0=Q_1=1$ 的次态翻转; Q_3 在 $Q_0=Q_1=Q_2=1$ 的次态翻转。当用 T 触发器实现 4 位同步二进制递增计数器时, 可根据上述激励条件, 确定每个 T 触发器的激励方程为

^① 表 6.5.5 是时序电路转换表的简化形式, 只给出在一系列时钟脉冲信号作用下电路状态的转换顺序, 若以某行状态作为现态, 下一行则为次态。

$$\begin{cases} T_0 = 1 \\ T_1 = Q_0 \\ T_2 = Q_1 Q_0 \\ T_3 = Q_2 Q_1 Q_0 \end{cases}$$

即除了触发器 FF_0 的 $T_0 = 1$ 外,其他触发器都是在比其低的各位均为 1 时准备好翻转条件。由此类推,可以扩展到更多的位数。当用 N 个 T 触发器构成 N 位同步二进制递增计数器时,各触发器的激励方程为

$$\begin{cases} T_0 = 1 \\ T_i = Q_{i-1} Q_{i-2} \cdots Q_1 Q_0 = \prod_{j=0}^{i-1} Q_j \quad (i=1, 2, \cdots, N-1) \end{cases} \quad (6.5.1)$$

表 6.5.5 4 位二进制计数器的转换表

计数顺序	递增计数	递减计数
	$Q_3 Q_2 Q_1 Q_0$	$Q_3 Q_2 Q_1 Q_0$
0	0 0 0 0	0 0 0 0
1	0 0 0 1	1 1 1 1
2	0 0 1 0	1 1 1 0
3	0 0 1 1	1 1 0 1
4	0 1 0 0	1 1 0 0
5	0 1 0 1	1 0 1 1
6	0 1 1 0	1 0 1 0
7	0 1 1 1	1 0 0 1
8	1 0 0 0	1 0 0 0
9	1 0 0 1	0 1 1 1
10	1 0 1 0	0 1 1 0
11	1 0 1 1	0 1 0 1
12	1 1 0 0	0 1 0 0
13	1 1 0 1	0 0 1 1
14	1 1 1 0	0 0 1 0
15	1 1 1 1	0 0 0 1
16	0 0 0 0	0 0 0 0

由表 6.5.5 递减计数一栏可以看出, Q_0 同样在每个计数脉冲沿到来时都要翻转一次;而其

他位都是在比其低的各位均为 0 时准备好翻转条件,在次态完成翻转,实现二进制计数的递减借位。当用 T 触发器实现 4 位同步二进制递减计数器时,激励方程为

$$\begin{cases} T_0 = 1 \\ T_1 = \bar{Q}_0 \\ T_2 = \bar{Q}_1 \bar{Q}_0 \\ T_3 = \bar{Q}_2 \bar{Q}_1 \bar{Q}_0 \end{cases}$$

对于 N 位同步二进制递减计数器,类似地有

$$\begin{cases} T_0 = 1 \\ T_i = \bar{Q}_{i-1} \bar{Q}_{i-2} \cdots \bar{Q}_1 \bar{Q}_0 = \prod_{j=0}^{i-1} \bar{Q}_j \quad (i = 1, 2, \cdots, N-1) \end{cases} \quad (6.5.2)$$

根据式(6.5.1)和式(6.5.2)可得由 T 触发器构成的 4 位同步二进制计数器逻辑图如图 6.5.9 所示。其中,图(a)为递增计数器,图(b)为递减计数器。

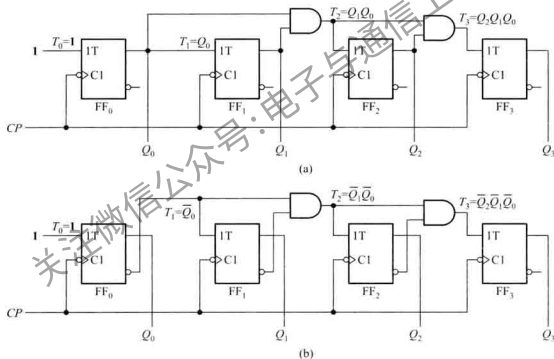


图 6.5.9 4 位同步二进制计数器

(a) 递增计数器 (b) 递减计数器

图 6.5.10 所示是图 6.5.9(a)所示计数器的时序图,其中虚线是考虑到触发器传输延迟时间 t_{pd} 的波形。由图 6.5.10 可知,在同步计数器中,由于计数脉冲 CP 同时作用于各触发器,所有触发器的状态更新几乎是同时进行的,都比计数脉冲沿的作用时间滞后一个 t_{pd} 。因此,输出状

态比异步二进制计数器稳定。对于同一逻辑系列,位数相同的计数器,同步计数器的工作速度一般高于异步计数器。

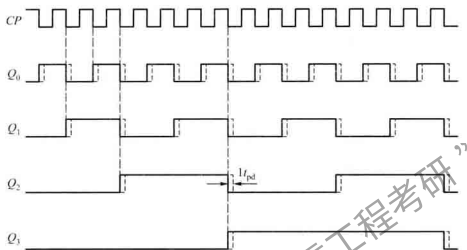


图 6.5.10 4 位同步二进制递增计数器时序图

当用 D 触发器实现二进制计数器时,可根据 5.5 节图 5.5.9(a) D 触发器与 T 触发器的关系,得到 D 触发器构成的 4 位二进制递增计数器如图 6.5.11 所示。其中,4 个点画线方框内为等效的 T 触发器。

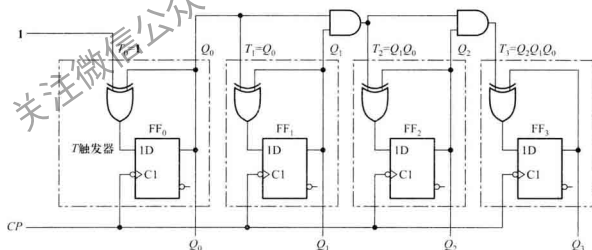


图 6.5.11 D 触发器构成的 4 位同步二进制递增计数器

② 同步计数器的进位方法

递增计数器的低位计满后激励高位触发器翻转的操作称为计数进位。图 6.5.11 所示电路

的进位信号是通过一系列串行的逻辑门逐位向高位传递的,这种计数器称为**同步串行计数器**。如果一个计数器中有 N 位触发器,将它们按二进制数位从低位向高位编号,最低位输出为 Q_0 ,那么, Q_0 的进位信号至少要经过 $N-2$ 个等效逻辑门的延迟,才可能到达最高位触发器的输入端。假设每个等效逻辑门的延迟时间相同,均为 t_{Gpl} ,那么计数器中组合电路的最大延迟时间为 $t_{\text{COMpl(max)}} = (N-2)t_{\text{Gpl}}$ 。显然, $t_{\text{COMpl(max)}}$ 将随着计数器的位数增多而增大。当计数器位数较多时,会降低电路的运行质量,或被迫降低电路的工作频率。

另一种计数器是**同步并行计数器**,电路结构如图 6.5.12 所示,其激励电路由 G_1 、 G_2 、 G_3 三个与门实现。和图 6.5.11 所示电路相比,虽然图 6.5.12 中各触发器的激励方程与前者相同,但后者在一次状态转换完成后,只需经过相当于一个等效逻辑门的传输延迟时间 t_{Gpl} ,就可将激励信号传递到所有高位触发器的 T 输入端,迎接下一个 CP 触发沿的到来。即无论计数器有多少位,电路的 $t_{\text{COMpl(max)}}$ 都等于一个等效逻辑门的传输延迟时间 t_{Gpl} 。因此,这种进位方式可以大大提高计数器的工作频率,位数越多,提高越明显。然而,随着计数器位数的增多,低位触发器的扇出和高位触发器激励电路的扇入会增多,例如,从图 6.5.12 所示电路中 Q_0 的扇出和 G_3 门的扇入就可看出这一点。

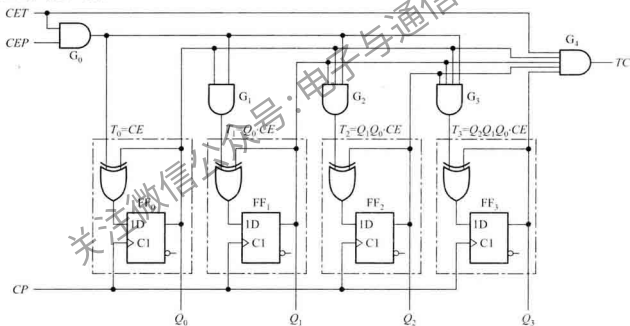


图 6.5.12 并行进位的同步 4 位二进制递增计数器

当计数器规模达到 4 位以上时,工程上常常采用折中处理的方案,将并行进位和串行进位相结合。方法是多将多位计数器分为若干组,组内的触发器采用并行进位,而组与组之间的进位则以串行实现。图 6.5.13 示出按这种方案实现 16 位同步二进制计数的一个实例。图中使用了 4 组图 6.5.12 所示电路,各组从低位到高位分别以 $C_0 \sim C_3$ 命名。需要注意的是每一组中串行进位允许信号 CET 、并行进位允许信号 CEP 和进位输出信号 TC 的连接方法。其中, $TC = Q_3 Q_2 Q_1 Q_0 \cdot CET$, 是

向高位组的进位信号。对于 C_0 组, CET 与 CEP 相连, 可作为整个计数器的使能输入 CE , 当 $CE=1$ 时允许计数。其他各组的 CEP 均接收 C_0 组计数器的进位信号 TC_0 ; C_1 组 CET 直接与计数允许输入相连, C_2 、 C_3 组的 CET 接收相邻低位组的进位信号。电路工作时, 只有在 C_0 组发出进位信号, 即 $TC_0=1$ 后, 才允许所有高位组内部进行并行进位操作; 高位各组的进位信息则通过各自的 G_4 直接上传。很容易看出, C_0 组 Q_0 的翻转信号, 只需经过该组的 G_4 和 C_3 组的 G_0 、 G_3 三个等效逻辑门的延迟, 历时大约 $3t_{Gpd}$, 就能抵达整个计数器的最高位触发器, 也就是 C_3 组的 FF_3 。进一步分析可以证明, 电路中所有触发器的激励电路延迟都不会大于 $3t_{Gpd}$, 即 $t_{COMpd(max)} \approx 3t_{Gpd}$ 。因而, 这种并、串行结合的进位方法既具有较短的进位延迟, 又同时兼顾了电路的复杂程度。

图 6.5.13 所示的进位方法, 对于设计大型同步计数器是具有普遍意义的, 包括 PLD 和 ASIC 的应用。例如, 在高速数据采集系统中, 经常需要用计数器对 1k 以上“字节(8 bit)”或“字(16 bit)”的存储器实现与取样同步的高速地址扫描, 以便及时将采集的数据顺序存入存储器。用于这种目的的二进制计数器, 位数一般都超过 10 位, 如果采用并、串行结合的进位方式, 则可提高电路工作的可靠性, 或充分发挥已有电路的资源。

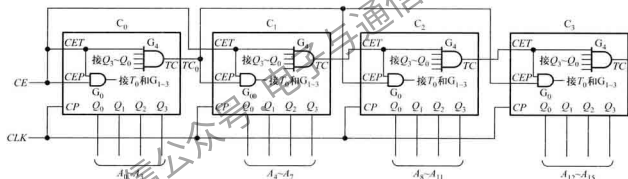


图 6.5.13 以图 6.5.12 所示电路实现的并行进位和串行进位结合的 16 位同步二进制计数器

③ 异步清零和同步并行置数

图 6.5.14 所示电路是功能更为完备的 4 位同步二进制计数器的逻辑图。

在许多应用中, 需要计数器从零状态开始工作, 为此, 在图 6.5.14 所示电路中增加了直接清零输入端 $\overline{CR}$ 。可以看到, 清零信号 $\overline{CR}$ 直接作用于 4 个触发器的置 0 端。当异步的 $\overline{CR}$ 为低电平时, 将使电路中所有触发器无条件置 0, 而不管其他输入端是何状态 (包括时钟信号 CP), 因此, $\overline{CR}=0$ 对计数器状态有优先级最高的控制权, 其他各种操作都是在 $\overline{CR}=1$ 的条件下才能执行。

计数器应用时, 有时需要将计数器预置为某种状态, 然后再从该状态开始计数。图 6.5.14 所示电路以图 6.5.12 为基础, 在每个 D 触发器的输入端前插入一个 2 选 1 数据选择器, 以选择从数据输入端 D_N 置入数据, 还是执行正常的计数功能, 并用一个公共置数控制端 $\overline{PE}$ 进行控制。

由图 6.5.14 看出, $D_3D_2D_1D_0$ 为 4 位并行数据输入端, 当 $\overline{PE}=0$ 时, 数据选择器选择并行数据送至相应触发器的 D 输入端, 这样, 在下一个 CP 沿到来时, 输入端 $D_3D_2D_1D_0$ 的数据被并行置入各触发器。这种在时钟脉冲作用下的并行置数称为同步置数。而当 $\overline{PE}$ 恢复高电平后, 4 个 2 选 1 数据选择器选择相应异或门的输出信号送至触发器 D 输入端, 计数器恢复正常计数功能, 于是, 可在预置数据的基础上继续递增计数。

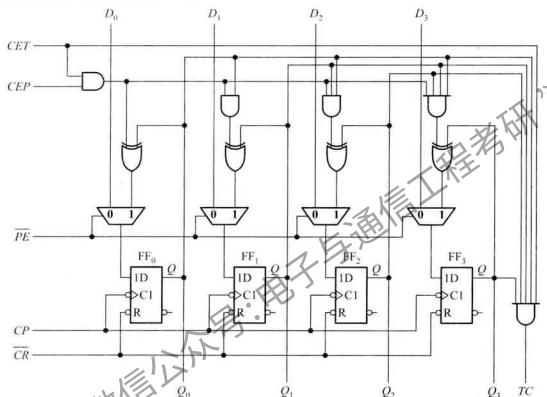


图 6.5.14 具有清零和并行置数功能的 4 位同步二进制递增计数器

由图 6.5.14 看出, $\overline{PE}=0$ 具有次高优先级, 仅低于 $\overline{CR}=0$, 计数和保持状态的操作都要求 $\overline{PE}=1$ 。

④ 典型集成同步二进制计数器

74LVC161 是一种典型的高性能、低功耗 CMOS 4 位同步二进制递增计数器, 芯片的逻辑功能和实现原理与图 6.5.14 所示逻辑图相同。它的时钟最高工作频率可达 200MHz。图 6.5.15 所示为 74LVC161 的引脚功能图, 所有引脚名称与图 6.5.14 所示输入/输出信号线一致。表 6.5.6 示出它的功能表。

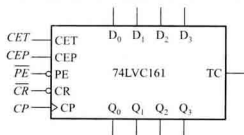


图 6.5.15 74LVC161 引脚功能图

表 6.5.6 74LVC161 的功能表

输 入									输 出				
清零 $\overline{CR}$	预置 $\overline{PE}$	使能 $\overline{CEP}$ $\overline{CET}$		时钟 CP	预置数据输入				Q_3	Q_2	Q_1	Q_0	进位 TC
L	X	X	X	X	X	X	X	X	L	L	L	L	L
H	L	X	X	$\uparrow$	D_3^*	D_2^*	D_1^*	D_0^*	D_3	D_2	D_1	D_0	#
H	H	L	X	X	X	X	X	X	保 持		保 持		#
H	H	X	L	X	X	X	X	X	保 持		保 持		L
H	H	H	H	$\uparrow$	X	X	X	X	计 数		计 数		#

注: D_n^* 表示 CP 脉冲上升沿之前瞬间 D_n 的电平。# 表示只有当 $Q_3Q_2Q_1Q_0 \cdot \overline{CET} = 1$ (二进制体系) 时, TC 输出为高电平, 其余均为低电平。

74LVC161 的典型时序图如图 6.5.16 所示。图中, 当清零信号 $\overline{CR} = 0$ 时, 各触发器置 0。当 $\overline{CR} = 1$ 时, 若 $\overline{PE} = 0$, 在下一个时钟脉冲上升沿到来后, 各触发器的输出状态与预置的输入数据相同。在 $\overline{CR} = \overline{PE} = 1$ 的条件下, 若 $\overline{CEP} = \overline{CET} = 1$, 则电路处于计数状态。图中从预置的 1100 开始计数, 直到 $\overline{CEP} \cdot \overline{CET} = 0$, 计数状态结束。此后处于禁止计数的保持状态: $Q_3Q_2Q_1Q_0 = 0010$ 。进位信号 TC 只有在 $Q_3Q_2Q_1Q_0 = 1111$ 且 $\overline{CET} = 1$ 时输出为 1, 其余时间均为 0。

与 74LVC161 工作原理和功能相似的集成计数器还有 74LVC163, 其差别仅在于清零方式不同, 后者为同步清零, 即清零信号 $\overline{CR}$ 的低电平必须等待下一个 CP 沿作用后才能完成清零操作。另外, 与它们结构类似的计数器还有 74LVC160 和 74LVC162, 这两种计数器是 8421 BCD 计数器, 前者为异步清零, 后者为同步清零。

2. 其他模数的计数器

除了模为 2ⁿ 的二进制计数器外, 工程上还经常需要很多其他模数的计数器。这类计数器的状态中都或多或少存在无效状态, 它们也分为同步和异步, 递增、递减和可逆等各种类型。其中, 最常用的是 BCD 计数器。

(1) 异步 BCD 计数器

在例 6.4.2 中曾分析过一个具有 000 ~ 100 五个连续二进制编码状态的异步计数器。如果在该电路基础上增加一级触发器, 便可构成异步 BCD 计数器, 如图 6.5.17 所示。为了应用的灵活性, 除清零信号 $\overline{CR}$ 外, 这个触发器和五状态计数器的输入、输出端均是独立引出的。

例 6.5.1 将图 6.5.17 所示电路按以下两种方式连接: ① $\overline{CP}_0$ 接计数脉冲信号; 将 Q_0 与 $\overline{CP}_1$ 相连。② $\overline{CP}_1$ 接计数脉冲信号, 将 Q_3 与 $\overline{CP}_0$ 相连。试分析它们的逻辑输出状态。

解: 按①方式连接时, 计数脉冲先进行二分频, 然后进行五分频。从 0000 状态开始, 依次分析, 得到的转换表如表 6.5.7 左边所示。 $Q_3Q_2Q_1Q_0$ 输出为 8421 BCD 码, 构成 8421 BCD 计

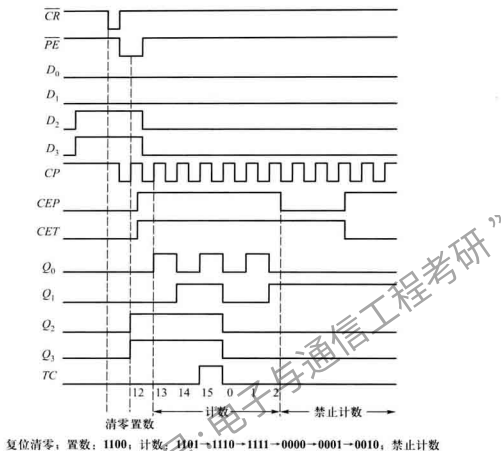


图 6.5.16 74LVC161 的典型时序图

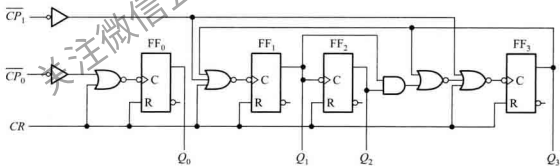


图 6.5.17 一种异步 BCD 计数器的逻辑图

数器。

按②方式连接时，计数脉冲先进行五分频，然后再二分频。分析得到的转换表列于表 6.5.7 右边。可以看出， Q_0 的权值等于 5， Q_3 、 Q_2 、 Q_1 的权值分别为 4、2、1，这种编码称为 5421 BCD 码。因此，电路构成 5421 BCD 计数器。

表 6.5.7 例 6.5.1 的两种连接方式的转换表

计数顺序	连接方式①(8421 码)				连接方式②(5421 码)			
	Q_3	Q_2	Q_1	Q_0	Q_0	Q_3	Q_2	Q_1
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	0
3	0	0	1	1	0	0	1	1
4	0	1	0	0	0	1	0	0
5	0	1	0	1	1	0	0	0
6	0	1	1	0	1	0	0	1
7	0	1	1	1	1	0	1	0
8	1	0	0	0	1	0	1	1
9	1	0	0	1	1	1	0	0

(2) 用集成计数器构成任意模数计数器

任意模数的计数器可以用厂家定型的集成计数器产品外加适当的电路连接而成。用模 m 的集成计数器构成模 n 计数器时,如果 $m > n$,则只需一个模 m 集成计数器;如果 $m < n$,则要用多个模 m 计数器来构成。下面结合例题分别介绍这两种情况的实现方法。

例 6.5.2 用 74LVC161 构成模 9 递增计数器。

解:模 9 计数器有 9 个状态,而 74 LVC 161 在计数过程中有 16 个状态。因此属于 $m > n$ 的情况。如果设法跳过多余的 7 个状态,则可实现模 9 计数器。通常用两种方法实现,即反馈清零法和反馈置数法。

① 反馈清零法

反馈清零法适用于有清零输入端的集成计数器。74LVC161 具有异步清零功能,在其计数过程中,不管它的输出处于哪一状态,只要在异步清零输入端加一低电平,使 $\overline{CR}=0$,74LVC161 的输出会立即从那个状态回到 0000 状态。清零信号消失后($\overline{CR}=1$),74LVC161 又从 0000 状态开始重新计数。

图 6.5.18(a)所示的模 9 计数器,就是借助 74LVC161 的异步清零功能实现的。图 6.5.18(b)是其主循环状态图。由图可知,74LVC161 从 0000 状态开始计数,当第九个 CP 脉冲上升沿到达时,输出 $Q_3Q_2Q_1Q_0=1001$,通过一个与非门译码后,反馈给 $\overline{CR}$ 端一个清零信号,立即使 $Q_3Q_2Q_1Q_0$ 返回到 0000 状态。此刻,产生清零信号的条件已消失, $\overline{CR}$ 端随之变为高电平,74LVC161 重新从 0000 状态开始新的计数周期。这样就跳过了 1001~1111 七个状态,构成模 9

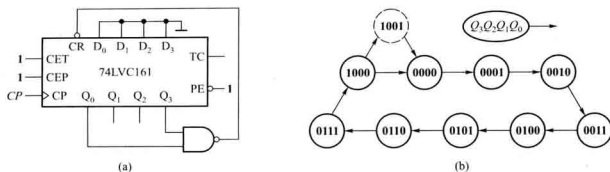


图 6.5.18 用反馈清零法将 74LVC161 接成模 9 计数器

(a) 电路图 (b) 主循环状态图

计数器。需要说明,电路是在进入 **1001** 状态后才被置成 **0000** 的,因此在主循环状态图中用虚线圈表示 **1001** 状态。虽然该状态存在的时间很短,且处于 **0000** 状态所占的时钟周期内,但是,如果 Q_0 端通过某种方式另接有其他时序电路,则应充分考虑 $Q_0=1$ 的窄脉冲是否会引起错误的逻辑输出,例如触发器被误触发等故障。

具有同步清零功能的模 m 集成计数器也可用反馈清零法构成模 n 计数器。这里不再举例,读者可自行分析两者的差异。

② 反馈置数法

反馈置数法适用于具有预置数功能的集成计数器。对于具有同步预置功能的计数器而言,在其计数过程中,可以将它输出的任何一个状态通过译码,产生一个预置控制信号反馈至预置控制端,在下一个 CP 脉冲作用后,就会把数据输入端 D_3, D_2, D_1, D_0 的状态置入计数器。预置控制信号消失后,计数器就从被置入的状态开始重新计数。

图 6.5.19(a) 和图 6.5.20 所示电路,都是借助 74LVC161 的同步预置功能,采用反馈置数法构成模 9 递增计数器的。其中图 6.5.19(a) 所示电路的接法是把输出 $Q_3Q_2Q_1Q_0=1000$ 的状态经译码产生预置信号 **0** 反馈至 $\overline{PE}$ 端,在下一个 CP 脉冲上升沿到达时置入 **0000** 状态。图 6.5.19(b) 是图 6.5.19(a) 所示电路的主循环状态图。其中 **0001** ~ **1000** 这 8 个状态是

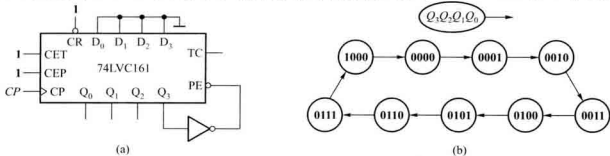


图 6.5.19 用反馈置数法将 74LVC161 接成模 9 计数器

(a) 电路图 (b) 主循环状态图

74LVC161 进行加 1 计数实现的, **0000** 是由反馈(同步)置数得到的。由此可以推知, 在图 6.5.19(a) 中, 反馈置数操作可在 74LVC161 计数循环状态 **0000** ~ **1111** 中的任何一个状态下进行。例如可将 $Q_3 Q_2 Q_1 Q_0 = 1111$ 状态的译码信号加至 $\overline{PE}$ 端, 这时, 数据输入端应接为 $D_3 D_2 D_1 D_0 = 0111$, 计数器将在 **0111** ~ **1111** 九个状态间循环。

图 6.5.20 所示电路的接法是将 74LVC161 计数到 **1111** 状态时产生的进位信号反相后, 反馈到预置端。预置数据输入端应置成 **0111** 状态。该电路从 **0111** 状态开始加 1 计数, 输入第 8 个 CP 脉冲后到达 **1111** 状态, 此时 $TC = Q_3 \cdot Q_2 \cdot Q_1 \cdot Q_0 \cdot CET = 1, \overline{PE} = 0$, 在第 9 个 CP 脉冲作用后, $Q_3 Q_2 Q_1 Q_0$ 被置成 **0111** 状态, 同时使 $TC = 0, \overline{PE} = 1$ 。新的计数周期又从 **0111** 开始。

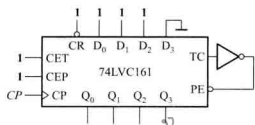


图 6.5.20 另一种反馈置数法的电路

具有异步置数功能的模 m 集成计数器也可用反馈置数法构成模 n 计数器。读者可自行分析它与上述同步置数的差异。

例 6.5.3 用图 6.5.17 所示异步 BCD 计数器构成模 24 计数器。

解: 运用反馈清零法实现。因为 BCD 计数器 $M=10$, 小于 24, 所以需要两个 BCD 计数器 C_0 和 C_1 实现。先将 C_0 和 C_1 分别接成 8421 BCD 计数器, 然后将它们级联, 接成模 100 计数器, 其中 C_1 构成十位数计数器, C_0 构成个位数计数器, 其电路如图 6.5.21 所示。在此基础上, 借助与门译码和计数器异步清零功能, 将 C_0 的 Q_2 和 C_1 的 Q_1 分别接至与门的输入端。工作时, 在第 24 个计数脉冲作用后, 十位数计数器输出为 **0010**, 个位数计数器输出为 **0100** (十进制数 24), C_1 的 Q_1 与 C_0 的 Q_2 同时为 1, 使与门输出高电平。它作用在计数器 C_0 和 C_1 的清零端 CR (高电平有效), 使计数器立即返回到 **0000 0000** 状态。状态 **0010 0100** 仅在瞬间出现一下, 计数器的有效状态为 24 个, 形成模 24 计数器。

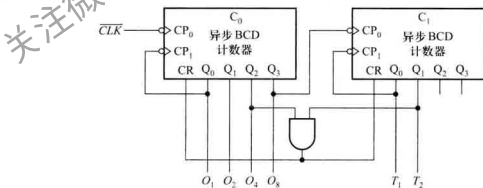


图 6.5.21 例 6.5.3 的电路图

这种连接方式可称为整体反馈清零法, 其原理与 $m > n$ 时的反馈清零法相同。也可以用具有预置数据功能的集成计数器, 采取整体反馈置数的方法构成模 24 计数器, 其原理与 $m > n$ 时的反

锁置数法相似。读者可自行分析或设计。

(3) 环形计数器

① 基本环形计数器

如果将图 6.5.2 中移位寄存器的 $D_{s0}(Q_3)$ 与 D_{s1} 相连, 则构成**环形计数器**, 如图 6.5.22(a) 所示。若事先通过 $\overline{PE}$ 端施加低电平脉冲, 在 4 个触发器内置入数据 $Q_3Q_2Q_1Q_0 = 0001$, 那么环形计数器在 CP 脉冲作用下, 将会有如图 6.5.22(b) 所示的 4 个状态, 于是, 电路成为模 4 计数器。图 6.5.22(c) 所示为在 4 个 CP 脉冲作用下的波形。可以看出, 这种计数器不必译码就能直接输出 4 个状态的译码信号, 亦不存在普通译码电路输出易出现的竞争-冒险现象。

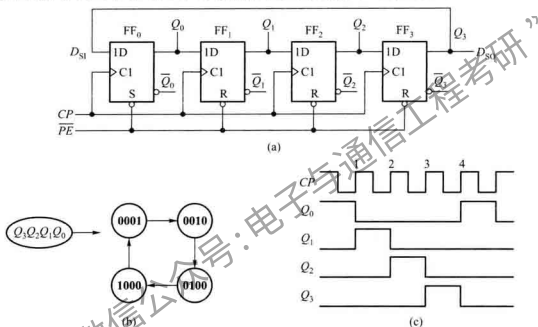


图 6.5.22 基本环形计数器的电路和状态变化

(a) 模 4 环形计数器 (b) 状态图 (c) 波形图

② 扭环形计数器

上述基本环形计数器的状态利用率不高, 4 个触发器只有 4 个计数状态。若将图 6.5.2 所示电路中的 $\overline{Q}_3$ 与 D_{s1} 相连, 则构成**扭环形计数器**, 亦称为约翰逊计数器 (Johnson counter), 电路的状态将增加一倍。图 6.5.23(a) 所示是 5 个触发器构成的扭环形模 10 计数器, 图 6.5.23(b) 所示是其状态图, 表 6.5.8 列出了它的十个状态以及对各状态译码的逻辑表达式。可以看出, 它的译码电路将十分简单。由于每一次时钟脉冲触发沿到来后只有一个触发器改变状态, 所以译码输出不会出现竞争-冒险。图 6.5.23(a) 中附加的逻辑门使电路能够自校正, 请读者自行验证。

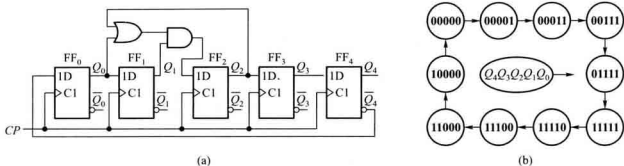


图 6.5.23 模 10 扭环计数器

(a) 逻辑图 (b) 状态图

表 6.5.8 图 6.5.23 所示扭环计数器的十个状态及其译码

计数顺序	Q_4	Q_3	Q_2	Q_1	Q_0	状态译码
0	0	0	0	0	0	$\overline{Q_4}\overline{Q_0}$
1	0	0	0	0	1	$\overline{Q_4}Q_0$
2	0	0	0	1	1	$\overline{Q_4}Q_1$
3	0	0	1	1	1	$\overline{Q_4}Q_2$
4	0	1	1	1	1	$\overline{Q_4}Q_3$
5	1	1	1	1	1	Q_4Q_0
6	1	1	1	1	0	$Q_4\overline{Q_0}$
7	1	1	1	0	0	$Q_4\overline{Q_1}$
8	1	1	0	0	0	$Q_4\overline{Q_2}$
9	1	0	0	0	0	$Q_4\overline{Q_3}$

74HC4017 是一种典型的模 10 扭环计数器,芯片中含有计数器的十个有效状态的译码电路,从引脚可直接得到 $D_0 \sim D_9$ 无竞争-冒险的译码输出。

复习思考题

- 6.5.1 电平敏感的锁存器和脉冲边沿敏感的寄存器在数据寄存操作上有何不同?
- 6.5.2 用 D 锁存器能否构成移位寄存器或计数器? 为什么?
- 6.5.3 如何用双向移位寄存器实现二进制数据的 $\times 2^n$ 和 $\div 2^n$ 运算?

6.5.4 同步和异步二进制计数器各有什么特点? 在需要高速计数并对计数值译码时应采用哪种计数器?

6.5.5 电子钟表中的石英晶体振荡器一般采用 32.768 kHz 的振荡频率, 需要使用多少位二进制计数器分频才能得到周期为 1s 的秒信号?

6.5.6 如何用具有同步清零功能的二进制计数器采取反馈清零法来构成模 n 计数器?

6.5.7 试用 10 个二输入端与门为图 6.5.23(a) 所示电路设计十个状态的译码电路, 并说明为什么它们的输出不会出现竞争-冒险。

6.6 简单的时序可编程逻辑器件 GAL

目前应用的时序可编程逻辑器件(Sequential Programmable Logic Device, SPLD)主要有 GAL、CPLD 和 FPGA 等。工程设计中, 应根据实际需求的简繁, 选择使用 GAL、CPLD 或 FPGA 等不同类型的时序 PLD。

本节将讨论的 GAL, 其集成度在 1000 个门以下, 属于简单、低密度型时序 PLD。对这种简单 PLD 的结构、工作原理和设计过程的理解, 将有助于更复杂的 PLD 的原理理解和应用。有关 CPLD 和 FPGA 的详细讨论将在第 8 章进行。

6.6.1 GAL 的结构

这里以较简单的 GAL16V8 为例, 说明 GAL 器件的结构。

1. 电路组成

图 6.6.1 为 GAL16V8 的逻辑结构图, 它主要由以下五部分组成:

① 由 8 组与门 $A_1 \sim A_8$ 共 8×8 个与门构成的可编程与逻辑阵列, 形成 64 个乘积项, 32 根列线通过编程可接至其中任意与门的输入端。32 根列线连接 16 对两两互补的变量, 其中 8 对接输入缓冲器的输出(列线编号为 0-1, 4-5, 8-9, 12-13, 16-17, 20-21, 24-25, 28-29), 另外 8 对接反馈/输入缓冲器的输出 $FB_1 \sim FB_8$ 。

② 8 个输出逻辑宏单元(Output Logic Macro Cell, OLMC)OLMC12 ~ OLMC19, 每个单元中都包括固定的或逻辑阵列。

③ 10 个输入缓冲器, 固定接信号输入端, 其中 8 路(引脚 $P_2 \sim P_9$)缓冲器 $IB_2 \sim IB_8$ 输出的互补信号直接引入与逻辑阵列, 而引脚 P_1 可作为系统时钟 CLK 的输入端, 引脚 P_{11} 输入的信号可作为输出三态缓冲器的系统使能控制信号 OE 。

④ 8 个连接 OLMC 的输出三态缓冲器 $OB_1 \sim OB_8$ 。若三态缓冲器配置为输出高阻状态, 所连接的相应引脚亦能作为输入端。

⑤ 8 个反馈/输入缓冲器 $FB_1 \sim FB_8$, 它们的输入信号来自各 OLMC, 其互补输出信号馈送到与逻辑阵列。

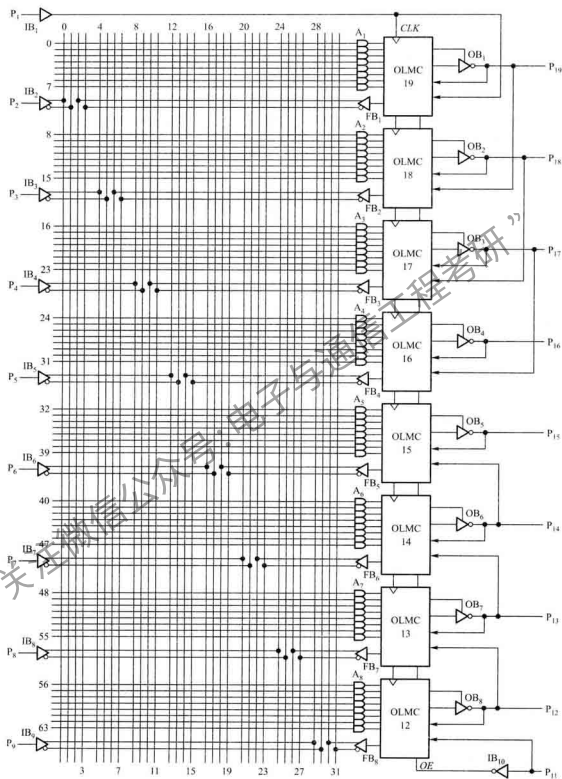


图 6.6.1 GAL16V8 的逻辑结构图

2. 与阵列编程单元

图 6.6.1 中 64 个与门的输入引线都和标有 0~31 的 32 根列线交叉, 因此每个与门有 32 个输入端。在每个交叉点上都设有 $E^2\text{CMOS}$ (Electrical Erasable CMOS, $E^2\text{CMOS}$) 编程单元。它采用了 4.5.1 节介绍的 Flotox 管。实现对乘积项 (Product Term, PT) 编程的原理电路如图 6.6.2 所示。图中左边是实际电路, 只画出了 $\bar{A}$ 、 B 、 C 三根列线, 省略了与它们互补的列线, 且每根列线只带了一个编程单元。图中, T_1 、 T_3 、 T_5 为逻辑功能管, T_2 、 T_4 、 T_6 、 T_7 为 Flotox 编程管。编程时, 编程电压加在 Flotox 管栅极对其进行编程 (未画编程电路); 在非编程状态时, Flotox 管的栅极不加逻辑电压, 列线上的逻辑电压加在 T_1 、 T_3 、 T_5 管的栅极。

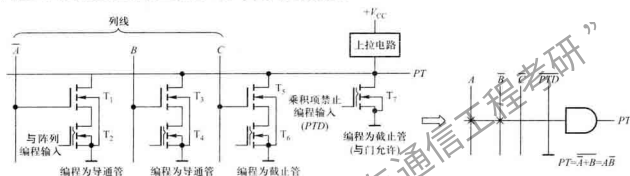


图 6.6.2 编程单元的原理电路

图 6.6.2 的电路图中, 因为 T_7 编程为截止管, 不影响 PT 输出, 故乘积项处于有效状态; 且因 T_6 也编程为截止管, 所以列线 C 上的逻辑电平与输出 PT 无关。而 T_2 和 T_4 编程为导通管, 则 T_1 和 T_3 构成或非门 (PT 端有电平上拉电路), 于是得到 $PT = \bar{A} + B = A \bar{B}$ 。此例中, 因为 A 、 $\bar{B}$ 和 PT 之间有编程连接, 所以在图 6.6.2 右边的简化逻辑图中, 相应的交叉点以 $\times$ 标记。需要注意的是左、右两图中输入变量是互补的。由于所有引入与阵列的变量全部是成对的互补变量, 所以, 这样安排电路并没有增加与阵列的复杂性。当该乘积项是冗余项时, T_7 应编程为导通管, 使 $PT = 0$, 则该乘积项输出为 0, 不再影响它所驱动的或逻辑, 表明它被禁止。

除了与阵列编程单元外, GAL 中的其他编程单元也都采用如图 6.6.2 中 T_7 那样的 $E^2\text{CMOS}$ 电路, 可编程为导通管, 也可以编程为截止管。需要时, 可用电擦除方式消除已编程的结构配置, 然后再重新编程。这给时序电路的设计、调试和修改带来极大的方便。

6.6.2 GAL 中的输出逻辑宏单元

1. 从组合 PAL 到寄存器型 PAL

如前所述, 时序电路由组合电路和存储单元组成。在第 4 章中已讨论过组合 PLD, 若在与-或阵列和输出缓冲电路之间插入触发器, 则组成如图 6.6.3 所示的电路, 构成时序可编程逻辑电路。图中, 触发器的 D 输入端被一组与-或阵列的输出所激励, 其 Q 端通过三态缓冲门接输出引线, 而 $\bar{Q}$ 端输出经互补缓冲电路将触发器的状态信息反馈到与逻辑阵列的可编程接点。这样一

种结构,不仅使 D 触发器可以寄存与-或逻辑阵列的输出状态,而且通过含有状态信息的乘积项 PT ,为触发器提供了从现态向次态转换的条件,可方便地组成不同逻辑功能的时序电路。在早期的“寄存器型 PAL”中,就集成了多组类似于图 6.6.3 所示的电路。其中,各三态缓冲门使能端都连接在同一输出使能线上,受外部输入的使能信号 OE 控制。各触发器的时钟信号输入也都连接到同一公共时钟 CLK 上,使一组触发器能同时更新状态,构成同步时序电路。

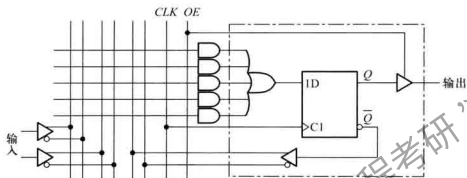


图 6.6.3 寄存器型 PAL 的输出电路

2. 在寄存器型 PAL 基础上改进的 GAL

GAL 是在寄存器型 PAL 基础上发展起来的增强型器件,其内部逻辑仍采用组合 PLD 的与-或结构,但在输出电路方面做了改进。在 GAL 中,每一组可编程的与门阵列驱动的是一个包括或门在内的输出逻辑宏单元 OLMC,它使电路的通用性更强,输出方式更灵活。GAL16V8 中 OLMC 的逻辑结构如图 6.6.4 所示。对照图 6.6.3 可以看出,该 OLMC 是在图 6.6.3 中点画线框内所示输出电路的基础上,增加了若干数据选择器 MUX 和一些门电路,电路设计者可根据需要将它们编程配置为不同结构,以实现多种结构的组合电路和时序电路。

3. OLMC 的电路结构和功能

图 6.6.4 中点画线框内所示为 OLMC 内部电路,主要由 1 个上升沿触发的 D 触发器,4 个数据选择器 MUX,一个固定的 8 输入端或门以及一些门电路组成。图中标注的变量分为两大类:一类是与其他 OLMC 共用的变量,如 CLK 、 OE 、 AC_0 等;另一类属于该 OLMC 专用或来自相邻 OLMC 的变量,如 $AC_1(n)$ 、 $XOR(n)$ 、 $OR(n)$ 、 $D(n)$ 、 $I/O(n)$ 、 $AC_1(m)$ 、 $I/O(m)$ 等,其中括号中的 n 表示该 OLMC 的编号,而 m 则是与 n 相邻的一个 OLMC 的编号。图中,“ Φ ”表示 E²CMOS 编程单元,通过编程来确定相应的逻辑变量为 0 还是 1。一旦对芯片编程完毕,它们的逻辑值就不可更改,除非将芯片整体擦除后重新编程。

(1) 输入电路

一个 OLMC 的或门与同一芯片中其他 OLMC 的或门构成了 GAL 的或逻辑阵列,再和输入的与门阵列一起构成基本的与-或组合逻辑阵列。OLMC 中或门的第 2~8 输入端分别直接和与阵列的一个与门输出端相连,即接收一个乘积项,而第 1 输入端接乘积项选择器 PT MUX 的输出。两个由编程单元确定逻辑值的变量 AC_0 和 $AC_1(n)$ 控制着 PT MUX 和三态控制选择器 TS MUX。

除了 OLMC12 和 OLMC19 两个输出逻辑宏单元有些特殊外,当编程使 $\overline{AC_0} \cdot \overline{AC_1(n)} = 1$ 时,第 1 乘积项通过 PT MUX 连接到或门的第 1 输入端,此时或门有 8 个乘积项输入,否则或门的第 1 输入端接 0,第 1 乘积项对或门的输出没有影响。在 $AC_0 = AC_1(n) = 1$ 时,第 1 乘积项将通过 TS MUX 接至输出三态缓冲器的使能端,当该乘积项为 1 时,三态缓冲器使能。

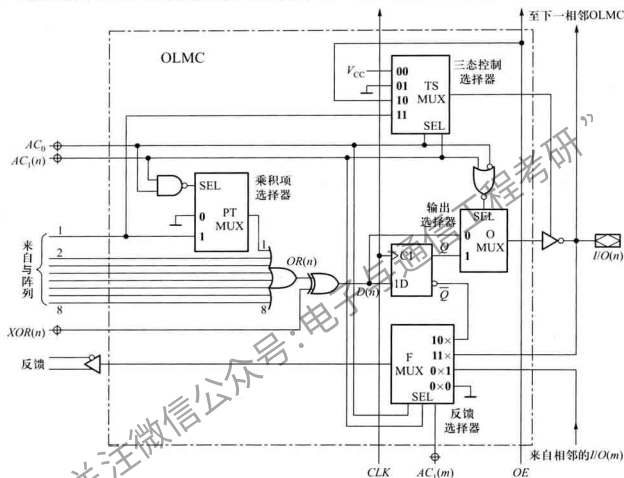


图 6.6.4 GAL16V8 中的 OLMC

注: $\oplus$ 表示 E²CMOS 编程单元, $\times$ 表示无关位

(2) 原变量/非变量输出控制电路

异或门将或门输出信号 $OR(n)$ 与 $XOR(n)$ 进行异或运算后,激励 D 触发器或直接输出。

$XOR(n)$ 的逻辑值由编程单元确定。对于异或门有 $D(n) = \overline{OR(n)} \cdot \overline{XOR(n)} + OR(n) \cdot XOR(n)$, 若 $XOR(n) = 0$, 则 $D(n) = OR(n)$, 若 $XOR(n) = 1$, 则 $D(n) = \overline{OR(n)}$ 。可见,通过对 $XOR(n)$ 编程,便可控制输出信号等于或门的输出,或者为或门输出的非。

(3) 输出电路

输出选择器 O MUX 是 2 选 1 数据选择器,用于选择输出信号的来源。当 AC_0 和 $AC_1(n)$ 满

足 $\overline{AC_0} + AC_1(n) = 0$ 时, 选择异或门输出, 其信号直接送达输出缓冲器, 意味着 OLMC 是组合电路输出; 反之, 当 $\overline{AC_0} + AC_1(n) = 1$ 时, O MUX 选择 D 触发器的 Q 端, 异或门的输出信号必须经过 D 触发器寄存, 才到达输出缓冲器, 即构建成时序电路输出。

输出三态缓冲器的控制信号来源由三态选择器 TS MUX 选择, 它是一个 4 选 1 数据选择器, 4 个数据输入端受 AC_0 和 $AC_1(n)$ 的控制, 可分别选择 V_{CC} 、地、外部输入 OE 或内部与门阵列的第 1 乘积项作为输出三态缓冲器的控制信号, 如表 6.6.1 所示。

表 6.6.1 TS MUX 的控制功能

控制信号		TS MUX 的输出	输出三态缓冲器 工作状态
AC_0	$AC_1(n)$		
0	0	1 (V_{CC})	使能
0	1	0 (地)	高阻
1	0	OE	OE=1: 使能 OE=0: 高阻
1	1	第 1 乘积项	第 1 乘积项=1: 使能 第 1 乘积项=0: 高阻

(4) 反馈电路

反馈电路包括反馈选择器 F MUX 和反馈/输入缓冲器, 它将不同来源的信号反馈到与门阵列。F MUX 是一个 4 选 1 数据选择器, 根据 3 个控制信号 AC_0 、 $AC_1(n)$ 和 $AC_1(m)$ 的编程逻辑值, 分别选择来自地(逻辑 0)、相邻 OLMC 的输出 $I/O(m)$ 、本级 OLMC 输出 $I/O(n)$ 或本级 D 触发器的 Q 信号送至与门阵列。上述 3 个控制信号的每项选择编码中, 都包含了一位逻辑值可 0、可 1 的无关位。

下面的例题分别列举了一个组合电路和一个时序电路的 OLMC 配置实例。

例 6.6.1 将编程单元 AC_0 、 $AC_1(n)$ 编程为 ① $AC_0 = AC_1(n) = 1$ 和 ② $AC_0 = 1$ 、 $AC_1(n) = 0$, 试画出编程后 OLMC 的功能逻辑图。

解: ① 当 $AC_0 = AC_1(n) = 1$ 时, PT MUX 选择接地, 第 1 个乘积项通过 TS MUX 接输出三态缓冲器的使能端, O MUX 将输出缓冲器直接与异或门输出相连, F MUX 将 $I/O(n)$ 接至反馈/输入缓冲器。得到的功能逻辑图如图 6.6.5 所示。此时 OLMC 配置为组合电路。

② 当 $AC_0 = 1$ 、 $AC_1(n) = 0$ 时, PT MUX 选择第 1 个乘积项接入或门, TS MUX 选择 OE 控制输出缓冲器使能, O MUX 选择 D 触发器的 Q 输出, F MUX 选择将触发器的 $\bar{Q}$ 接至反馈/输入缓冲器, 得到的功能逻辑图如图 6.6.6 所示。芯片中若有几个同样配置的 OLMC, 它们共用一个时钟信号 CLK, 可构成同步时序电路。

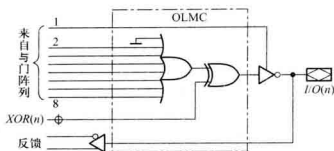


图 6.6.5 OLMC 配置为组合电路

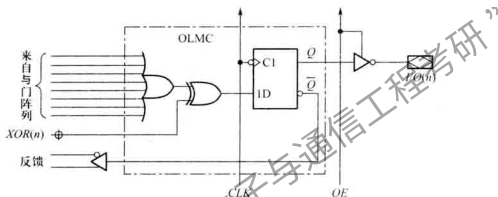


图 6.6.6 OLMC 配置为时序电路

6.6.3 GAL 的结构控制字

如前所述,通过编程可将 OLMC 配置成各种不同的逻辑功能。由图 6.6.4 看出,每个 OLMC 有 2 个编程单元 $AC_1(n)$ 和 $XOR(n)$,另外有一个全局编程单元 AC_0 。而 GAL16V8 共有 8 个 OLMC (参见图 6.6.1),所以共有 8 个 $AC_1(n)$ 编程单元和 8 个 $XOR(n)$ 编程单元。另外, GAL16V8 中的 64 个与门每个都有一个乘积项禁止编程单元 (参见图 6.6.2),共 64 个。将这些编程单元组合在一起,构成所谓的结构控制字,每个编程单元占据其中 1 位。GAL16V8 的结构控制字共有 82 位,它们的定义如图 6.6.7 所示。图中 $XOR(n)$ 和 $AC_1(n)$ 字段下面的数字为 n 的值,表示所控制的 OLMC 编号。

除前面已介绍的编程单元外,结构控制字中还有一个关键的同步控制单元 SYN 。当 $SYN=1$ 时,芯片中所有的 OLMC 均配置为组合输出模式;若 $SYN=0$,则 OLMC 可配置为寄存器输出模式 (即时序电路模式),亦可配置为组合输出,但至少有一个 OLMC 配置为寄存器输出模式。此外,对于 OLMC(12) 和 OLMC(19) 两个宏单元, $\overline{SYN}$ 代替 AC_0 , SYN 代替 $AC_1(m)$,这使得 $SYN=1$ 时, P_1 脚和 P_{11} 脚成为输入端,将输入信号通过这两个 OLMC 中的 F MUX 和反馈/输入缓冲器引入与逻辑阵列,使这两引脚的逻辑功能与 $P_2 \sim P_9$ 引脚相同。当初这种安排是为了兼容早期的很多

PAL 器件。

通过对结构控制字的编程,便可确定 GAL 的工作模式。

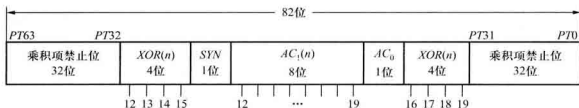


图 6.6.7 GAL16V8 的结构控制字

除了结构控制字外, GAL16V8 内部还附有 1 个加密编程单元和 64 位电子标签编程单元。前者被编程后就禁止对门阵列作进一步的编程或读出验证,以防止未经允许的改写和保护电路的知识产权。加密编程单元只有在全部编程单元被整体擦除后才被擦除,但电子标签记录的信息不受加密单元状态的影响,加密后仍随时可以读出。电子标签可供产品制造者记录各种识别信息,如产品制造商识别码、编程日期、线路形式识别码、产品序列码等,以便识别型号相同而编程后功能不同的 GAL 芯片,以及进行产品的质量跟踪。另外,还有一个整体擦除编程单元,在编程周期对该单元执行擦除命令,则将前述全部编程单元整体擦除,包括电子标签和加密单元,芯片返回到初始状态。

为了实现对 GAL16V8 的编程和回读验证编程状态,芯片中设置有编程硬件电路。对 GAL 的编程需要在开发系统(包括硬件编程器和软件)的控制下完成。其开发过程将在 8.3 节统一介绍。

从上述有关讨论可以看出, GAL16V8 虽然比此前的 PAL 等可编程器件的功能有很大提高,但是,它的应用仍受到 OLMC 有限几种工作模式的限制,而且其电路规模仅相当于中规模集成电路芯片,当需要实现更大规模的数字系统时,应当选用集成度更高、编程功能更强的可编程逻辑器件,如 CPLD、FPGA 等。

复习思考题

- 6.6.1 GAL 中的输出三态缓冲器可由哪几个信号控制?
- 6.6.2 在 GAL 的 OLMC 中可以将哪几个信号反馈到与门阵列中?
- 6.6.3 在时序电路设计中, GAL 的应用受到哪些局限?

6.7 用 Verilog HDL 描述时序逻辑电路

在 4.6 节和 5.6 节分别介绍了用 Verilog 描述组合电路和触发器的方法,本节将通过一些例

题讨论时序逻辑电路的行为描述方法。

6.7.1 移位寄存器的 Verilog 建模

例 6.7.1 根据 6.5.4 所示的功能表, 试用 Verilog 语言描述一个 4 位双向移位寄存器的行为。

解:

```
module shift4x194_beh( //Verilog 2001,2005 syntax
    input S1,S0,           //选择输入端口声明
    input Dsl,Dsr,         //串行数据输入
    input CP,CR,           //时钟和清零输入
    input [3:0]D,          //并行数据输入
    output reg[3:0]Q       //输出端口及变量的数据类型声明
);
    always @ (posedge CP,negedge CR)//Verilog 2001,2005 syntax
    if( ~ CR ) Q<= 4'b0000; //实现异步清零功能
    else
        case( {S1,S0} )
            2'b00:Q<=Q;      //输出保持不变
            2'b01:Q<= {Q[2:0],Dsr}; //右移
            2'b10:Q<= {Dsl,Q[3:1]}; //左移
            2'b11:Q<=D;      //并行置数
        endcase
    endmodule
```

程序首先声明了模块的输入、输出端口变量,然后使用 **always** 语句结构描述了模块完成的 5 种逻辑功能:异步置零、同步置数、左移、右移和保持原状态不变。其功能与图 6.5.6 所示的 74HC/HCT194 类似。

当清零信号 CR 跳变到低电平时,寄存器的输出被异步置 0;当 CR=1 时,与时钟信号有关的 4 种功能由 **case** 语句中的两个选择输入信号 S1、S0 决定(在 **case** 后面 S1、S0 被拼接成两位矢量)。移位由串行输入和 3 个触发器的输出拼接起来进行描述,例如:

$$Q <= \{Dsl, Q[3:1]\};$$

说明了左移操作,即在时钟信号 CP 上升沿到来时,将左移输入端 Dsl 的数据直接传给输出 Q[3],而 CP 上升沿到来之前触发器输出端的数据(现态)左移 1 位, Q[3:1] 传给 Q[2:0](即 Q[3]→Q[2], Q[2]→Q[1], Q[1]→Q[0]),于是,完成将数据左移 1 位的操作。

注意,例 6.7.1 中所描述的右移和左移方向与图 6.5.6 中的逻辑图一致,与计算机原理中的规定相反。

6.7.2 计数器的 Verilog 建模

下面介绍同步二进制计数器、异步二进制计数器和非二进制计数器的 Verilog 建模。

1. 同步二进制计数器

例 6.7.2 根据表 6.5.6 所示的功能表,试用 Verilog 语言描述一个 4 位计数器的行为。

解: 4 位计数器的 Verilog 语言描述如下所示。在该模块中混合使用了 **assign** 语句和 **always** 语句,**assign** 语句描述了组合电路中由与门产生的使能控制信号 CE(中间节点)和进位输出信号 TC,当计数器计数到最大值 15 时,TC=1。

always 语句描述了计数器的逻辑功能,当 CR 信号跳变到低电平(由 **negedge** CR 描述)时,计数器的输出被置 0;否则,当 CR=1 时,在 CP 上升沿作用下,完成其他三种功能:同步置数、加 1 计数和保持原有状态不变。注意,**if** 语句隐含着各个控制信号 CR、PE、CE 操作的优先级别。逻辑综合工具根据该例的行为模型将产生图 6.5.13 所示的电路。

```

module counter74x161_beh(                                //Verilog 2001,2005 syntax
    input CEP,CET,PE,CP,CR,                            //输入端信号声明
    input [3:0]D,                                       //并行数据输入
    output TC,                                          //进位输出
    output reg [3:0]Q                                   //数据输出端口及变量的数据类型声明
);
    wire CE;                                           //中间变量声明
    assign CE=CEP & CET;                               //CE=1 时,计数器计数
    assign TC=CET & PE & (Q==4'b1111);               //产生进位输出信号
    always @ ( posedge CP,negedge CR)                 //Verilog 2001,2005 syntax
    if ( ~ CR) Q<= 4'b0000;                           //实现异步清零功能
    else if ( ~ PE) Q<= D;                             //PE=0,同步装入输入数据
    else if (CE) Q<= Q+1'b1;                           //加 1 计数
    else Q<= Q;                                         //输出保持不变
endmodule

```

例 6.7.3 试用 Verilog 语言描述一个带使能端和同步置数端的通用可逆计数器的行为。

解: 一个通用的可逆计数器 Verilog 代码如下所示。其中参数 n(默认设为 4)用来设置二进制计数器的位数。当 Up_down=1 时,整型变量 direction 为 1,计数器为递增计数;Up_down=0 时,direction 为-1,计数器为递减计数。

```

module updowncount_beh(                                //Verilog 2001,2005 syntax
    parameter n=4
)

```



```

(
input Load, Up_down, En, CP, //输入端口声明
input[ n-1:0 ] D, //并行数据输入端口声明
output reg[ n-1:0 ] Q //数据输出端口及变量的数据类型声明
);
integer direction; //声明一个整型变量 direction
always @ ( posedge CP)
begin
    if( Up_down)
        direction<= 1; //Up_down=1, 整型变量 direction 为 1
    else
        direction<= -1; //Up_down=0, 整型变量 direction 为-1
    if( Load)
        Q<= D; //Load=1, 同步装入输入数据
    else if( En)
        Q<= Q+direction; //实现加 1 或减 1 计数功能
    else
        Q<= Q; //输出保持不变
end
endmodule

```

2. 异步二进制计数器

例 6.7.4 根据图 6.5.7 所示原理图, 试用 Verilog 语言描述一个 4 位异步二进制计数器的行为。

解: 异步计数器的结构化描述如下。第一个模块通过 4 次调用第二个模块完成计数功能, 作为底层的第二个模块是带有异步置零功能的 D 触发器。

在第一个模块中, 第 1 个触发器 FF0 的时钟信号接外部输入 CP 端, 其输出 Q0 经反相(用 ~Q0 表示)后与 D 输入相连(在 FF0 中用 ~Q0 取代 D), 构成 T' 触发器。第 2 个触发器 FF1 的时钟信号接前一个触发器的输出端(用 Q0 取代 CP), 其输出 Q1 经反相后与 D 输入相连(在 FF1 中用 ~Q1 取代 D)。类似地, 将 4 个触发器级联在一起构成异步二进制计数器, 其原理图与图 6.5.7 类似。

```

module ripplecounter(
output Q0, Q1, Q2, Q3, //输出端口声明
input CP, CR //输入端口声明
);
D_FF FF0( Q0, ~Q0, CP, ~CR ); //实例化 D 触发器
D_FF FF1( Q1, ~Q1, Q0, ~CR );

```

```

D_FF FF2(Q2, ~Q2,Q1, ~CR);
D_FF FF3(Q3, ~Q3,Q2, ~CR);
endmodule

module D_FF(output reg Q,input D,CP,Rd );//D 触发器模块
always @( negedge CP,negedge Rd)
if( ~Rd)Q<=1'b0; //异步清零
else Q<=D; //Rd=1,将输入数据送到输出端口
endmodule

```

3. 非二进制计数器

例 6.7.5 试用 Verilog 语言描述一个带有使能端和异步清零端的同步模 10 计数器。

解：同步模 10 计数器的 Verilog 代码如下。当清零信号 CR 跳变到低电平(由 negedge CR 描述)时,计数器的输出被置 0;否则,当 CR=1,且使能信号 CE=1 时,在 CP 上升沿作用下,若计数值小于 9,则计数器的值加 1;若计数值大于或者等于 9,计数器的输出被置零。当 CR=1,但 CE=0 时,计数器保持原来的状态不变。注意,电路的功能描述与具体的硬件电路结构是无关的。

```

module m10_counter( //Verilog 2001,2005 syntax
output reg[3:0]Q, //输出端口及变量的数据类型声明
input CE,CP,CR //输入端口声明
);
always @( posedge CP,negedge CR)
if( ~CR)Q=4'b0000; //异步清零
else if(CE, //CE=1,由下面的 if-else 语句完成模 10 计数
begin
if(Q>=4'b1001)
Q<=4'b0000;
else Q<=Q+1'b1;
end
else Q<=Q; //输出保持不变(默认情况)
endmodule

```

4. 分频器

例 6.7.6 假设有一个 50 MHz 时钟信号源,试用 Verilog 设计一个分频电路,以产生 1 Hz 的秒脉冲输出,要求输出信号的占空比为 50%。

解：设计一个模数为 25×10^6 的二进制递增计数器,其计数范围是 0~24999999,每当计数器计到最大值时,输出信号翻转一次,即可产生 1 Hz 的秒脉冲。其程序如下。

```

module Divider50MHz(
parameter CLK_Freq = 50000000,           //50MHz 时钟输入
parameter OUT_Freq = 1,                   //1 Hz 时钟输出
)
(
input      CR,CLK_50M,                     //输入端口声明
output reg CLK_1HzOut                      //输出端口及变量的数据类型声明
);
reg[24:0] Count_DIV;                       //内部节点
always @ (posedge CLK_50M or negedge CR)
begin
if(! CR)  begin
        CLK_1HzOut<=0;                     //输出信号被异步清零
        Count_DIV<=0;                     //分频器的输出被异步清零
    end
else begin
        if( Count_DIV<( CLK_Freq/2 * OUT_Freq) )
            Count_DIV<= Count_DIV+1'b1;   //分频计数器增1 计数
        else begin
            Count_DIV    <=0;               //分频器的输出被清零
            CLK_1HzOut    <= ~CLK_50M;      //输出信号取反
        end
    end
end
endmodule

```

6.7.3 状态图的 Verilog 建模

描述状态图的方法很多,下面通过两个例题进行说明。

例 6.7.7 试用 Verilog 语言描述图 6.7.1 所示状态图的行为。

解: 图 6.7.1 所示状态图的行为描述如下。程序首先对电路的输入、输出、时钟以及清零信号进行声明,对保存电路状态值的触发器用标识符 state(中间节点)进行声明,并使用参数定义语句 **parameter** 定义了电路的四种状态,即 $S0=2'b00$ 、 $S1=2'b01$ 、 $S2=2'b10$ 和 $S3=2'b11$ 。

接着,用一个 **always** 语句描述时钟控制的状态转换以及电路的次态逻辑。在异步复位信号 CR 为 0 时,电路进入初始状态 S0;在没有复位信号且时钟信号 CP 上升沿到来时,根据电路当前状态和输入信号,电路转换到下一个状态,这是用 **case** 语句进行描述的,在 **case** 语句的多个分支中包含了每个状态的行为。

最后用 **assign** 语句描述了电路的输出,该电路的输出与电路现态相同。注意,在 CP 信号的两个上升沿之间,输入信号 Data 的变化对输出信号是没有影响的。

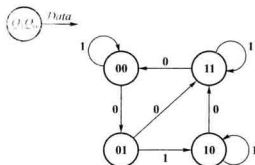


图 6.7.1 一个穆尔型时序电路的状态图

```

module MooreMachine(           //Verilog 2001,2005 syntax
input Data,CP,CR,             //输入端口声明
output [1:0] Q                //输出端口声明
);
    reg [1:0] state; //说明两个寄存器变量 state[1]和state[0],记忆电路现态
    parameter S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11; //电路的状态定义
    always @ (posedge CP,negedge CR)
    begin
        if( ~ CR) state<=S0; //CR=0,让电路进入初始状态 S0
        else
            case(state) //电路状态转换
            S0:if( ~ Data) state<=S1;
            S1:if( Data) state<=S2;else state<=S3;
            S2:if( ~ Data) state<=S3;
            S3:if( ~ Data) state<=S0;
            endcase
        end
        assign Q=state; //电路的输出与电路的现态相同
    endmodule

```

上例描述状态图的方法仅仅用了一条 **always** 语句,看起来非常简单,但电路结构却不够清晰,电路的现态和次态均用变量 **state** 表示,并且该方法对米利型时序电路不适用,因为米利型电路输出可能直接受到与时钟无任何关系的输入信号的作用。建议用两条 **always** 语句对状态图建模,下面举例说明。

例 6.7.8 图 6.3.7 是用于检测连续输入序列 110 的状态图,试用 Verilog 描述该状态图所

表示的行为。

解：该状态图的 Verilog 描述如下。

```

module Mealy_sequence_detector( //Verilog 2001,2005 syntax
    input A,CP,CR,           //输入端口声明
    output reg Y             //输出端口及变量的数据类型声明
);
    reg[1:0] current_state,next_state;//中间变量声明
    parameter S0=2'b00,S1=2'b01,S2=2'b11;//定义状态
    always @ ( negedge CP,negedge CR)//触发器状态转换
    begin
        if ( ~ CR )    current_state<=S0; //让电路进入初始化状态 S0
        else current_state<=next_state;//电路发生状态转换(相当于 D 触发器 Q=D)
    end
    always @ ( current_state,A) //输出逻辑和次态逻辑描述
    begin
        case( current_state)
            Y=0;           //电路输出
            S0;begin next_state=( A == 1)? S1:S0;end
            S1;begin next_state=( A == 1)? S2:S0;end
            S2;if( A == 1)
                begin next_state<=S2;end
            else
                begin Y=1;next_state<=S0;end
            default;begin next_state<=S0;end
        endcase
    end
endmodule

```

电路功能描述使用了两条并行执行的 **always** 语句,通过公用变量(next_state) 相互交换信息。第一条 **always** 语句使用边沿触发事件描述电路的同步时序逻辑部分,第二条 **always** 语句使用电平敏感事件描述组合逻辑部分。

第一条 **always** 语句说明了异步复位到初始状态 S0 和同步时钟控制的操作,语句

```
current_state<= next_state;
```

仅在时钟 CP 的下降沿被执行,这意味着第二条 **always** 语句内部 next_state 值的变化会在时钟 CP 下降沿到来时被传送给 current_state。

第二条 **always** 语句把现态 current_state 和输入数据 Data 作为敏感变量,只要其中的任何一个变量发生变化,就会执行顺序语句块内部的 **case** 语句,跟在 **case** 语句后面的各分支项说明了

图 6.3.7 中状态的转换以及输出信号。

注意,在 Mealy 型电路中,当电路处于任何给定的状态时,如果输入信号 A 发生变化,则输出信号 Y 也会跟着变化。

综上所述,利用 Verilog HDL 语言描述状态图主要包含四部分内容:

(1) 利用参数定义语句 **parameter** 描述状态机中各个状态的名称,并指定状态编码。例如,对序列检测器的状态分配可以使用最简单的自然二进制码,其描述如下:

```
parameter S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11;
```

```
或者,parameter[1:0]S0=2'b00,S1=2'b01,S2=2'b10,S3=2'b11;
```

注意,在语法上使用 $S2=3$ 的形式定义状态也是正确的,但存储“3”这个整数至少需要使用 32 位寄存器,而存储 $2'b11$ 只需要两位寄存器,所以例题中使用的定义方式更好一些^②。上面第二种方式的定义中,明确地指出使用两个状态触发器,对逻辑综合更为有利。

(2) 使用 **always** 语句描述时钟控制的同步时序逻辑部分,实现电路状态存储。

(3) 使用 **always** 语句和 **case** 语句(也可以采用 **if-else** 等价语句)描述电路的组合逻辑部分,实现状态转换逻辑。

(4) 描述状态机的输出逻辑。

6.7.4 数字钟的 Verilog 建模

例 6.7.9 设计一个具有时、分、秒计时的数字钟电路,按 24 小时制计时。要求:

(1) 输出时、分、秒的 8421BCD 码,计时输入脉冲频率为 1Hz;

(2) 具有分、时校正功能,校正输入脉冲频率为 1Hz;

(3) 采用分层次、分模块的方法,用 Verilog 语言进行设计。

解: (1) 设计分析

数字钟的组成框图如图 6.7.2 所示,它由两个 60 进制计数器、1 个 24 进制计数器和两个 2 选 1 选择器共 5 个模块构成,三个计数器公用一个时钟信号 CP,为同步 8421BCD 码输出的计数器,在秒、分、小时计数器的输出端接上虚线框内的译码显示电路,即可显示出数码(即时间)。

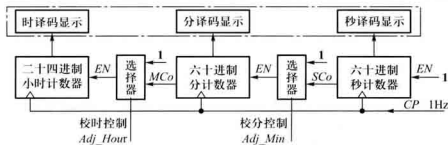


图 6.7.2 数字钟组成框图

图中两个选择器分别用于选择分钟计数器和小时计数器的使能控制信号。对时间进行校正

时,在控制端(*Adj_Hour*、*Adj_Min*)的作用下,使能信号接高电平,此时每来1个时钟信号,计数器加1计数,从而实现对小时和分钟的校正。正常计时时,使能信号来自于低位计数器的输出,即秒钟计数器计到59秒时,产生输出信号(*SCo*=1)使分钟计数器加1,分钟、秒钟计数器同时计到最大值(59分59秒)时,产生输出信号(*MCo*=1)使小时计数器加1。

(2) 逻辑设计

实现上述功能的 Verilog HDL 程序如下。整个程序分为两个层次4个模块,其层次结构如图6.7.3所示。底层由3个模块组成,六进制计数器模块(*counter6.v*)、十进制计数器模块(*counter10.v*)和24进制计数器模块(*counter24.v*),顶层有1个模块(*top_clock.v*),顶层模块调用底层的3个模块完成数字钟的计时功能。其中,底层的六进制计数器模块和十进制计数器模块分别被调用两次,构成六十进制的秒钟计数器和分钟计数器。

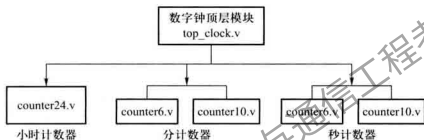


图 6.7.3 数字钟程序的层次结构图

```

// ***** 时钟顶层模块 (top_clock.v) *****
module top_clock( //Verilog 2001,2005 module port syntax
    input CP,nCR,EN,Adj_Min,Adj_Hour, //输入端口变量声明
    output [7:0] Hour,Minute,Second //输出端口变量声明
);
    supply1 Vdd;
    wire SecH_EN,MinL_EN,MinH_EN,Hour_EN; //中间节点变量声明
    //60 进制秒计数器:调用 10 进制和 6 进制底层模块构成
    counter10 U1(Second[3:0],nCR,EN,CP); //秒计数器个位
    counter6 U2(Second[7:4],nCR,SecH_EN,CP); //秒计数器十位
    //产生分钟计数器使能信号。Adj_Min=1,校正分钟;Adj_Min=0,分钟正常计时
    assign MinL_EN=Adj_Min? Vdd:(Second==8'h59);
    assign MinH_EN=(Adj_Min && (Minute[3:0]==4'h9)) || (Minute[3:0]==4'h9) && (Second==8'h59);

    //60 进制分钟计数器:调用 10 进制和 6 进制底层模块构成
    counter10 U3(Minute[3:0],nCR,MinL_EN,CP); //分钟计数器个位
    counter6 U4(Minute[7:4],nCR,MinH_EN,CP); //分钟计数器十位
    //产生小时计数器使能信号。Adj_Hour=1,校正小时;Adj_Hour=0,小时正常计时

```

```

    assign Hour_EN = Adj_Hour? Vdd : ( ( Minute == 8'h59 ) && ( Second == 8'h59 ) );
    //24 进制小时计数器:调用 24 进制底层模块构成
    counter24 U5( Hour[ 7:4 ], Hour[ 3:0 ], nCR, Hour_EN, CP );//小时计数器
endmodule

//***** counter10. v( BCD:0 ~ 9 ) *****
module counter10(
    input CP, nCR, EN,
    output reg[ 3:0 ] Q
);
    always @ ( posedge CP, negedge nCR )
    begin
        if ( ~ nCR )            Q <= 4'h0000;        //nCR=0, 计数器被异步清零
        else if ( ~ EN )        Q <= Q;              //EN=0, 暂停计数
        else if ( Q == 4'h1001 ) Q <= 4'h0000;
        else                    Q <= Q + 1'b1;        //计数器增 1 计数
    end
endmodule

//***** counter6. v( BCD:0 ~ 5 ) *****
module counter6(
    input CP, nCR, EN,
    output reg[ 3:0 ] Q
);
    always @ ( posedge CP, negedge nCR )
    begin
        if ( ~ nCR )            Q <= 4'h0000;        //nCR=0, 计数器被异步清零
        else if ( ~ EN )        Q <= Q;              //EN=0, 暂停计数
        else if ( Q == 4'h0101 ) Q <= 4'h0000;
        else                    Q <= Q + 1'b1;        //计数器增 1 计数
    end
endmodule

//***** counter24. v( BCD:0 ~ 23 ) *****
module counter24(
    input CP, nCR, EN,
    output reg[ 3:0 ] CntH, CntL,
);

```



```

always @ ( posedge CP, negedge nCR)
begin
    if( ~ nCR)
        | CntH, CntL | <= 8 ' b00;
    else if( ~ EN)
        | CntH, CntL | <= | CntH, CntL | ;
    else if( ( CntH>2) || ( CntL>9) || ( ( CntH = 2) && ( CntL>= 3) ) )
        | CntH, CntL | <= 8 ' b00; // 出错处理
    else if( ( CntH = 2) && ( CntL<3) ) // 小时 (20 ~ 23) 的计数
        begin CntH<= CntH; CntL<= CntL+1 ' b1; end
    else if( CntL = 9) // 小时十位计数
        begin CntH<= CntH+1 ' b1; CntL<= 4 ' b0000; end
    else // 小时个位计数
        begin CntH<= CntH; CntL<= CntL+1 ' b1; end
end
endmodule

```

复习思考题

6.7.1 试用 Verilog 描述图 6.5.1 所示的寄存器。

6.7.2 异步计数器和同步计数器的描述方式有何不同？

6.7.3 状态图的建模方法与前述计数器的建模有何区别？试用状态图的建模方法描述一个 4 位同步二进制递增计数器。

小 结

• 时序电路一般由组合电路和存储电路两部分构成。它们在任一时刻的输出不仅是当前输入信号的函数，而且还与电路原来的状态有关。时序电路可分为同步和异步两大类。逻辑方程组、转换表、状态表、状态图和时序图从不同方面表达了时序电路的逻辑功能，是分析和设计时序电路的主要工具。

• 同步时序电路是目前广泛应用的时序电路，是本章重点讨论的内容。同步时序电路的分析，首先按照给定电路列出各逻辑方程组、进而列出转换表或状态表、画出状态图和时序图，最后分析得到电路的逻辑功能。同步时序电路的设计，首先根据逻辑功能的需求，推导出原始状态图或原始状态表，有必要时需进行状态化简，继而对状态进行编码得到转换表，然后根据转换表导出激励方程组和输出方程组，最后画出逻辑图完成设计任务。

• 时序电路的功能、结构和种类繁多。本章仅对寄存器和计数器等几种典型的时序电路模块进行了较详细的讨论。应用这些电路模块，能设计出各种不同功能的电子系统。这些模块的

内部电路结构可作为设计其他逻辑电路的范例,也可作为初学者使用 HDL 以抽象方式描述时序电路时的实际映象。

● 本章分别在 6.6 节和 6.7 节介绍了简单时序可编程逻辑器件 GAL 和用 Verilog 描述时序电路的方法,它们是电子技术和计算机技术以及 EDA 技术高度发展的结果。这两节所讨论的方法和器件,尚需参考有关软件的详细文献资料并配合上机实验才能达到实际应用的程度。

习 题

6.1 时序逻辑电路的基本概念

6.1.1 已知一时序电路的状态表如表题 6.1.1 所示, A 为输入信号,试作出相应的状态图。

表题 6.1.1

S^n	S^{n+1}/Z	
	$A=0$	$A=1$
a	$d/1$	$b/0$
b	$d/1$	$c/0$
c	$d/1$	$a/0$
d	$b/1$	$c/0$

6.1.2 已知状态表如表题 6.1.2 所示,输入为 A_1A_0 ,试作出相应的状态图。

表题 6.1.2

S^n	S^{n+1}/Z			
	$A_1A_0=00$	$A_1A_0=01$	$A_1A_0=10$	$A_1A_0=11$
S_0	$S_0/0$	$S_1/0$	$S_3/0$	$S_2/1$
S_1	$S_1/0$	$S_2/1$	$S_3/1$	$S_0/0$
S_2	$S_2/0$	$S_1/0$	$S_3/0$	$S_3/0$
S_3	$S_3/0$	$S_0/1$	$S_2/0$	$S_2/0$

6.1.3 已知状态图如图题 6.1.3 所示,试列出它的转换表。

6.1.4 试画出 101 序列检测器的状态图,已知此检测器的输入序列、输出序列如下:

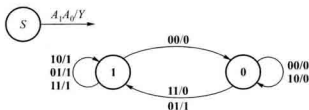
(1) 输入 A : 0 1 0 1 0 1 1 0 1

输出 Z : 0 0 0 1 0 1 0 0 1

(2) 输入 A : 0 1 0 1 0 1 1 0 1 0

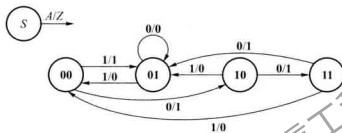
输出 Z : 0 0 0 1 0 0 0 0 1 0

6.1.5 图题 6.1.5 所示是某时序电路的状态图,设电路的初始状态为 01,当序列 $A=100110$ 自左至右输入



图题 6.1.3

时,求该电路输出 Z 的序列。



图题 6.1.5

6.1.6 已知某时序电路的状态表如表题 6.1.6 所示,输入为 1,试画出它的状态图。如果电路的初始状态为 b ,输入信号 A 依次是 **0 1 0 1 1 1 1**,试求其相应的输出。

表题 6.1.6

S^n	S^{n+1}/Z	
	$A=0$	$A=1$
a	$a/0$	$b/0$
b	$a/1$	$d/1$
c	$b/1$	$e/1$
d	$d/0$	$c/0$
e	$b/1$	$a/1$

6.1.7 已知某同步时序电路含有两个上升沿触发的 D 触发器,其激励方程组为

$$D_0 = X_2 X_1 + X_1 Q_0 + X_2 Q_0$$

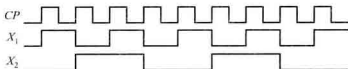
$$D_1 = X_1 \oplus X_2 \oplus Q_0$$

输出方程为

$$Z = Q_1$$

列出转换表,画出状态图,并分析其逻辑功能。若输入信号的波形如图题 6.1.7 所示,且电路的初始状态为 **00**,试画出 Q_1 、 Q_0 的波形。

6.1.8 已知转换表如表题 6.1.8 所示,若电路的初始状态为 $Q_1 Q_0 = 00$,输入信号 A 的波形如图题 6.1.8 所



图题 6.1.7

示,输出信号为 Z ,试画出 $Q_1 Q_0$ 的波形(设触发器为下降沿触发)。

表题 6.1.8

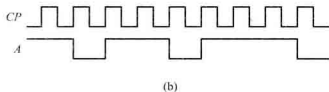
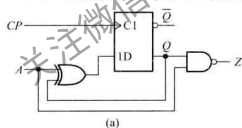
$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Z$	
	$A=0$	$A=1$
0 0	0 1/1	1 1/1
0 1	1 0/0	1 0/0
1 0	1 0/0	1 1/0
1 1	0 1/1	0 0/1



图题 6.1.8

6.2 同步时序逻辑电路的分析

6.2.1 试分析图题 6.2.1(a) 所示时序电路,列出转换表并画出状态图。设电路的初始状态为 0,试画出在图题 6.2.1(b) 所示波形作用下, Q 和 Z 的波形图。



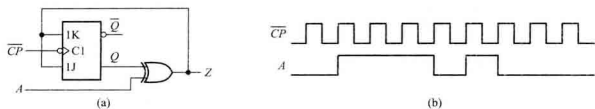
图题 6.2.1

6.2.2 试分析图题 6.2.2(a) 所示时序电路,列出转换表并画出状态图。设电路的初始状态为 0,画出在图题 6.2.2(b) 所示波形作用下, Q 和 Z 的波形图。

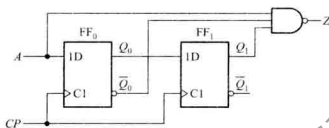
6.2.3 试分析图题 6.2.3 所示同步时序电路,画出状态图。

6.2.4 试分析图题 6.2.4 所示电路,写出它的激励方程组、转换方程组和输出方程,列出转换表并画出状态图。

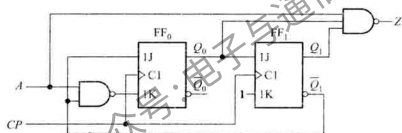
6.2.5 试分析图题 6.2.5 所示同步时序电路,写出各触发器的激励方程、电路的转换方程组和输出方程,列出转换表,画出状态图。



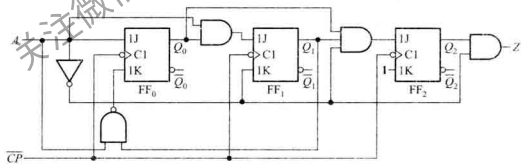
图题 6.2.2



图题 6.2.3

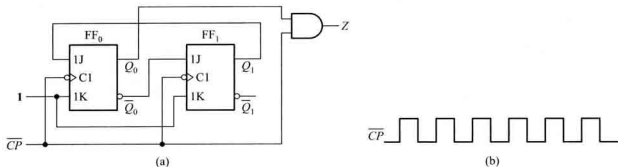


图题 6.2.4



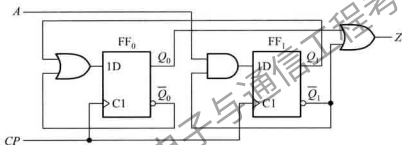
图题 6.2.5

6.2.6 试画出图题 6.2.6(a) 所示同步时序电路的状态图, 并画出对应于 CP 的 Q_1 、 Q_0 波形, 以及输出 Z 的波形, 设电路的初始状态为 00 。



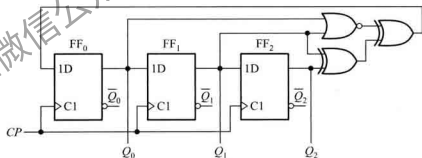
图题 6.2.6

6.2.7 试分析图题 6.2.7 所示同步时序电路, 写出激励方程组、转换方程组和输出方程, 列出转换表并画出状态图。



图题 6.2.7

6.2.8 试分析图题 6.2.8 所示同步时序电路, 写出激励方程组、转换方程组, 列出转换表并画出状态图。



图题 6.2.8

6.3 同步时序逻辑电路的设计

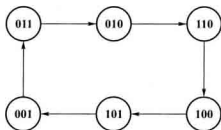
6.3.1 用 JK 触发器设计一同步时序电路, 其转换表如表题 6.3.1 所示。

表题 6.3.1

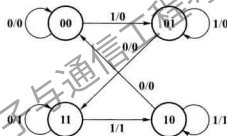
$Q_1^n Q_0^n$	$Q_1^{n+1} Q_0^{n+1} / Y$	
	$A = 0$	$A = 1$
00	01/0	11/0
01	10/0	00/0
10	11/0	01/0
11	00/1	10/1

6.3.2 某同步时序电路的状态图如图题 6.3.2 所示,试写出用 D 触发器设计时的最简激励方程组。

6.3.3 试用上升沿触发的 JK 触发器设计一同步时序电路,其状态图如图题 6.3.3 所示,要求电路使用的门电路最少。

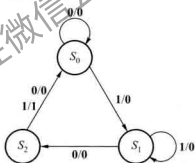


图题 6.3.2



图题 6.3.3

6.3.4 试用下降沿触发的 D 触发器设计一同步时序电路,其状态图如图题 6.3.4(a) 所示, S_0 、 S_1 、 S_2 的编码如图题 6.3.4(b) 的卡诺图所示。



(a)

	Q_0	
Q_1	S_0	S_1
	S_2	$\times$

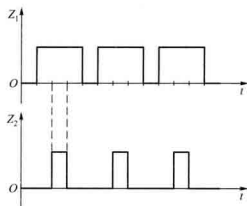
(b)

图题 6.3.4

6.3.5 试用下降沿触发的 JK 触发器和最少的门电路,实现图题 6.3.5 所示的 Z_1 和 Z_2 输出波形。

6.3.6 试用上升沿触发的 D 触发器设计一个 1101 序列检测器,输入为串行编码序列,输出为检出信号。

6.3.7 试用 D 触发器设计一同步时序电路,其状态表如表题 6.3.7 所示。



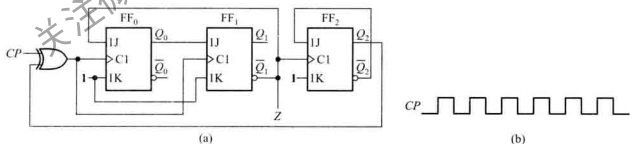
图题 6.3.5

表题 6.3.7

S^n	S^{n+1}		Z
	$A=0$	$A=1$	
a	b	d	0
b	c	b	0
c	b	a	1
d	a	c	0

6.4 异步时序逻辑电路的分析

6.4.1 一时序电路如图题 6.4.1(a) 所示, 试画出在 CP 作用下, Q_0 、 Q_1 、 Q_2 和 Z 端的波形, 设各触发器的初始状态均为 0。

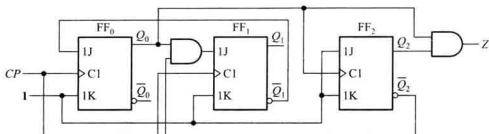


图题 6.4.1

6.4.2 分析图题 6.4.2 所示时序电路 [CP 脉冲同图题 6.4.1(b)]。

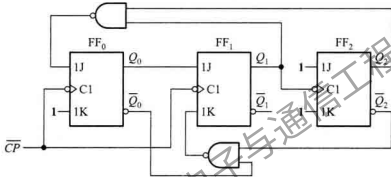
- (1) 写出各触发器的 CP 信号方程和激励方程;
- (2) 写出电路的转换方程组和输出方程;
- (3) 列出转换表并画出状态图;

(4) 画出电路的时序图。



图题 6.4.2

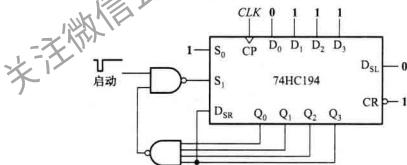
6.4.3 试分析图题 6.4.3 所示时序电路的逻辑功能。



图题 6.4.3

6.5 若干典型的时序逻辑电路

6.5.1 试画出图题 6.5.1 所示电路的输出 ($Q_3 \sim Q_0$) 波形, 并分析该电路的逻辑功能。

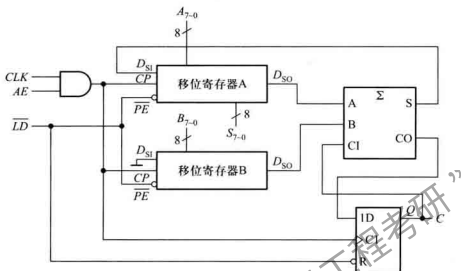


图题 6.5.1

6.5.2 试用两片 74HC194 构成 8 位双向移位寄存器。

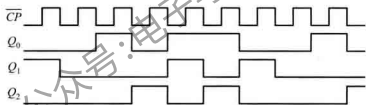
6.5.3 用一个全加器和一个 D 触发器及两个 8 位移位寄存器 A、B 构成的 8 位串行加法电路如图题 6.5.3 所示。图中, CLK 为时钟输入端; $\overline{LD}$ 为置数控制输入端, 当 $\overline{LD}=0$ 时, 8 位被加数 $A_{7..0}$ 和 8 位加数 $B_{7..0}$ 将分别进入移位寄存器 A 和 B; AE 为加运算控制端, 当 $AE=1$ 时, 进行串行加法运算, 输入 8 个时钟脉冲后恢复为 0; $S_{7..0}$ 为

8 位和输出端; C 为进位输出端。移位寄存器 A、B 的 CP 端为时钟输入端, $\overline{PE}$ 端为并行置数控制端, D_{SI} 和 D_{SO} 端分别为串行数据输入端、输出端。试分析电路的工作原理。



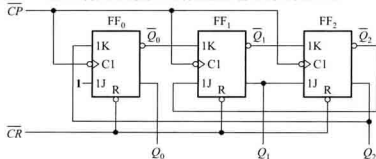
图题 6.5.3

6.5.4 在某计数器的输出端观察到如图题 6.5.4 所示的波形, 试确定该计数器的模。



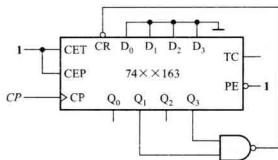
图题 6.5.4

- 6.5.5 试用下降沿触发的 JK 触发器组成 4 位异步二进制递减计数器, 画出逻辑图。
 6.5.6 试用下降沿触发的 D 触发器组成 4 位异步二进制递增计数器, 画出逻辑图。
 6.5.7 试用上升沿触发的 D 触发器及门电路组成 3 位同步二进制递增计数器, 画出逻辑图。
 6.5.8 试分析图题 6.5.8 所示计数器的模, 画出各触发器输出端的波形图。



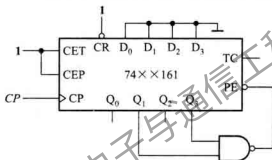
图题 6.5.8

能的 4 位同步二进制递增计数器,其他功能与 74×161 相同)



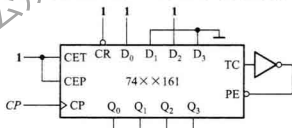
图题 6.5.14

6.5.15 试分析图题 6.5.15 所示计数器,画出它的状态图,并确定它的模。



图题 6.5.15

6.5.16 试分析图题 6.5.16 所示计数器,画出它的状态图,并确定它的模。



图题 6.5.16

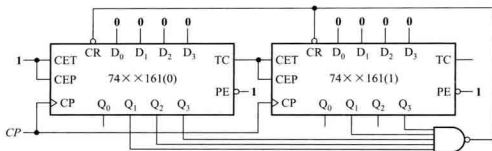
6.5.17 试用 74HC161 设计一个计数器,其计数序列以自然二进制数 1001 ~ 1111 顺序循环。

6.5.18 试分析图题 6.5.18 所示计数器,确定它的模。

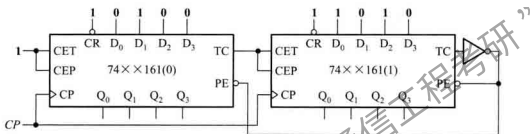
6.5.19 试分析图题 6.5.19 所示计数器,确定它的模。

6.5.20 试用 74×161 构成同步模 24 计数器,要求采用两种不同的方法。

6.5.21 试用 4 个具有复位功能的 D 触发器设计一个扭环型计数器,用复位方式将计数器初始状态置为 $Q_3Q_2Q_1Q_0 = 0000$,并用 8 个二输入端与门对它的 8 个计数状态译码,画出逻辑电路图。



图题 6.5.18



图题 6.5.19

6.6 简单的时序可编程逻辑器件 GAL

6.6.1 对于图 6.6.4 所示的 OLMC, 试画出当 $dC_0=1, AC_1(n)=1, XOR(n)=1$ 时的等效逻辑电路。

6.7 用 Verilog HDL 描述时序逻辑电路

6.7.1 根据图 6.5.2 所示的逻辑图, 试用 Verilog 语言描述 4 位移位寄存器的功能。然后用 Quartus II 软件进行逻辑功能仿真, 并给出仿真波形。

6.7.2 阅读下列程序, 说明它所完成的功能。

```
module shiftn(Q_PData, SerialData, Load, CP);
    input SerialData;           //Serial Data inputs
    input CP, Load;            //Clock and Load control
    input [n-1:0] PData;       //Parallel Data input
    output reg [n-1:0] Q;       //Register output
    parameter n = 8;
    integer k;
    always @ (posedge CP)
        if (Load) Q <= PData;   //Parallel load input
        else
            begin
                for (k = 0; k < n-1; k = k+1)
                    Q[k] <= Q[k+1];
                Q[n-1] <= SerialData;
            end
endmodule
```

end

endmodule

6.7.3 试用 Verilog 语言描述一个 4 位二进制可逆计数器的行为。要求如下:

(1) 电路具有 5 种功能,即异步清零、同步置数、递增计数、递减计数和保持原有状态不变。且要求计数器能输出进位信号和借位信号,即当计数器递增计数到最大值时,产生一个高电平有效的进位信号 Co ;当计数器递减计数到最小值 0 时,产生一个高电平有效的借位信号 Bo 。

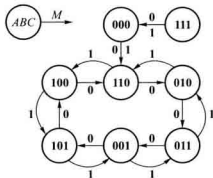
(2) 用 Quartus II 软件进行逻辑功能仿真,并给出仿真波形。

6.7.4 试用 Verilog 语言描述一个变模计数器,在 S 和 T 的控制下,实现同步模 5、模 8、模 10 和模 12 计数,其模数控制表如表题 6.7.4 所示,并要求具有异步清零和暂停计数的功能。然后用 Quartus II 软件进行逻辑功能仿真,并给出仿真波形。

表题 6.7.4 计数器的模数控制表

控制信号		模数
S	T	
0	0	模 5 计数
0	1	模 8 计数
1	0	模 10 计数
1	1	模 12 计数

6.7.5 某电路的状态图如图题 6.7.5 所示,图中, M 为控制变量,当 $M=0$ 时,电路按顺时针方向所指的状态进行转换;当 $M=1$ 时,则按逆时针方向进行状态转换。试用 Verilog 描述该电路的功能,并用 Quartus II 软件进行逻辑功能仿真,并给出仿真波形。



图题 6.7.5

6.7.6 设计一个序列检测器电路。功能是检测出串行输入数据 $Data$ 中的 4 位二进制序列 **0101**(自左至右输入),当检测到该序列时,输出 $Out=1$;没有检测到该序列时,输出 $Out=0$ 。要求:

(1) 给出电路的状态编码,画出状态图(注意考虑序列重叠的可能性,如 **010101**,相当于出现两个 **0101** 序列)。

(2) 用 JK 触发器和门电路来设计此电路。

(3) 用 Verilog 的行为描述方式描述该电路的功能。

(4) 然后用 Quartus II 软件进行逻辑功能仿真,并给出仿真波形。

6.7.7 试用 Verilog 的行为描述方式写出数字钟的小时时间计数器程序。要求如下:

(1) 计数器的功能是从 1 开始计数到 12,然后又从 1 开始,周而复始运行。计数器的输出为 8421 BCD 码。

(2) 要求该计数器带有复位端 CR 和计数控制端 EN。当 CR 为低电平时,计数器复位,其输出为 1;当 CR 和 EN 均为高电平时,计数器处于计数状态;当 CR 为高电平但 EN 为低电平时,计数器暂停计数。

(3) 然后用 Quartus II 软件进行逻辑功能仿真,并给出仿真波形。

6.7.8 在例 6.7.9 的基础上,将计时输入脉冲频率改为 1 kHz,要求增加以下功能:

(1) 具有仿电台整点报时功能。即每逢 59 分 51 秒、53 秒、55 秒、57 秒输出 500 Hz 低音频信号,在 59 分 59 秒时输出 1 kHz 高音频信号,输出音频信号的持续时间均为 1 s,高音频信号结束时,正好为整点。

(2) 增加闹钟功能,最长闹铃时间为 1 min。闹钟的闹铃时刻可以任意设置(只要求对小时、分钟进行设置);闹铃信号为 500 Hz 和 1 kHz 的方波信号,两种频率的信号交替输出,且均持续 1 s。设置一个停止闹铃的控制键,可以停止输出闹铃信号。

(3) 采用分层次、分模块的方法,用 Verilog 语言进行设计,并给出各模块的仿真波形图。

关注微信公众号:电子与通信工程考研